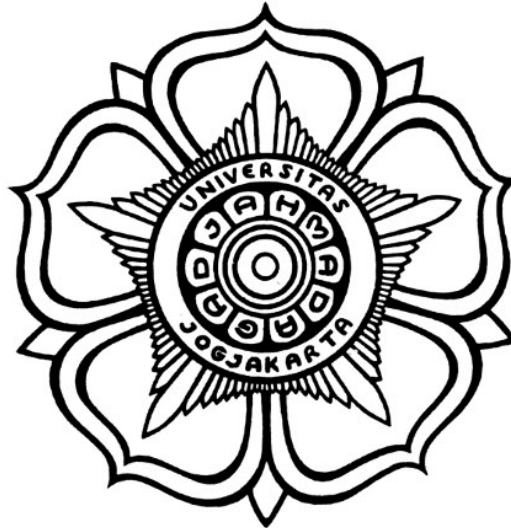


**RAYCASTED: A RAYCAST END-TO-END DETECTION MODEL
FOR LIGHTWEIGHT AND BOUNDARY-AWARE NUCLEI
DETECTION IN HISTOPATHOLOGY IMAGES**

SKRIPSI



**THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action**

Written by:

Danish Dhiaurrahman Ritonga
22/492936/TK/53966

BIOMEDICAL ENGINEERING PROGRAM

**DEPARTMENT OF ELECTRICAL ENGINEERING AND
INFORMATION TECHNOLOGY
FACULTY OF ENGINEERING UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

ENDORSEMENT PAGE

RAYCASTED: A RAYCAST END-TO-END DETECTION MODEL FOR LIGHTWEIGHT AND BOUNDARY-AWARE NUCLEI DETECTION IN HISTOPATHOLOGY IMAGES

THESIS

Proposed as A Requirement to Obtain
Undergraduate Degree (*Sarjana Teknik*)
in Department of Electrical Engineering and Information Technology
Faculty of Engineering
Universitas Gadjah Mada

Written by:

Danish Dhiaurrahman Ritonga
22/492936/TK/53966

Has been approved and endorsed

on

Supervisor I

Supervisor II

Dosen Pembimbing 1, S.T., M.Eng., PhD.
«NIP xxxxxx»

Dosen Pembimbing 2, S.T., M.Eng., PhD.
«NIP xxxxxx»

STATEMENT

Saya yang bertanda tangan di bawah ini :

Nama :
NIM :
Tahun terdaftar :
Program : Sarjana
Program Studi :
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Nama Mahasiswa
NIM

PAGE OF DEDICATION

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

PREFACE

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Orang 1 yang telah
2. Orang 2 yang telah
3. <isi dengan nama orang lainnya>

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi kita semua, aamiin.

Catatan: setiap nama yang dituliskan boleh disertai dengan alasan berterima kasih.

CONTENTS

ENDORSEMENT PAGE	ii
STATEMENT.....	iii
PAGE OF DEDICATION	iv
PREFACE.....	v
CONTENTS	vi
LIST OF TABLES.....	x
LIST OF FIGURES	xi
NOMENCLATURE AND ABBREVIATION	xii
INTISARI.....	xiv
ABSTRACT	xv
CHAPTER I Introduction	1
1.1 Background	1
1.2 Problem Statement.....	2
1.3 Objectives	2
1.4 Research Scope	3
1.5 Research Novelty	3
1.6 Research Contributions	3
1.7 Presentation Structure	4
CHAPTER II Tinjauan Pustaka dan Dasar Teori.....	5
2.1 Literature Review	5
2.1.1 StarDist	5
2.1.2 CPP-Net	5
2.1.3 HoVer-NeXt	5
2.1.4 CellPose	6
2.1.5 LSP-DETR	6
2.2 Theoretical Basis.....	7
2.2.1 Histopathological Foundations	7
2.2.1.1 Nuclear Morphology and Shape Characteristics	7
2.2.1.2 The Overlapping Nuclei Problem	8
2.2.1.3 Area-based Measurements	8
2.2.1.4 Cell Count-based Measurements.....	9
2.2.2 Digital Pathology Imaging.....	10
2.2.2.1 Tissue Slicing and Scanning	10
2.2.2.2 Histopathological Slides Staining.....	10
2.2.2.3 Beer-Lambert Law and Optical Density	11
2.2.2.4 Stain Vectors	11

2.2.2.5	Stain Vector Estimation Methods	12
2.2.2.6	Color Deconvolution	13
2.2.2.7	Stain Normalization	14
2.2.3	Segmentation Paradigms.....	14
2.2.3.1	Semantic Segmentation.....	14
2.2.3.2	Instance Segmentation.....	15
2.2.3.3	Limitations of Segmentation for Nuclei Analysis.....	15
2.2.4	Detection Paradigms	16
2.2.4.1	Anchor Grid.....	16
2.2.4.2	Matching Mechanisms in Object Detection	16
2.2.4.3	Feature Pyramid Networks	18
2.2.4.4	Non-Maximum Suppression	18
2.2.4.5	Limitations of Detection for Nuclei Analysis	19
2.2.5	Convolutional Neural Networks	20
2.2.5.1	Convolution and Feature Extraction	20
2.2.5.2	Receptive Field and Kernel Size	20
2.2.5.3	Depthwise Convolution.....	21
2.2.6	Discrete Wavelet Transform	21
2.2.6.1	Wavelet Decomposition Fundamentals	21
2.2.6.2	Discrete Wavelet Transform.....	22
2.2.6.3	Wavelet-Based Downsampling in CNNs	23
2.2.7	Star-Convex Object Representation	24
2.2.7.1	Geometric Definitions	24
2.2.7.2	Suitability for Nuclei Representation	25
2.2.8	End-to-End Detection with Bipartite Matching	25
2.2.8.1	Bipartite Matching: Hungarian Algorithm	25
2.2.8.2	Task-Aligned Matching.....	26
2.2.8.3	Dual-Label Assignment	26
2.2.9	Evaluation Metrics	27
2.2.9.1	Precision, Recall, and F1 Score	27
2.2.9.2	Mean Average Precision (mAP)	27
2.2.9.3	Panoptic Quality (PQ)	28
2.2.9.4	Aggregated Jaccard Index (AJI).....	30
CHAPTER III	Research Methodology	31
3.1	Datasets and Environmental Setup	31
3.1.1	Datasets.....	31
3.1.1.1	PanNuke.....	31
3.1.1.2	PUMA.....	31
3.1.1.3	panopTILs.....	31

3.1.1.4	MoNuSAC	32
3.1.2	Environmental Setup	32
3.2	Methods	32
3.2.1	Data Preprocessing	32
3.2.1.1	Ingestion Pipeline	32
3.2.1.1.1	Representation	32
3.2.1.1.2	Ingestion Design	37
3.2.1.2	Transform Pipeline	38
3.2.1.2.1	Profile Registration	38
3.2.1.2.2	Vector Estimation and Normalization	39
	Population-Level Stain Vector Estimation.	39
	Per-Tile Stain Normalization.	40
3.2.1.2.3	Size Standardization	41
	Chopping: Dynamic Overlap Distribution.	41
	Annotation-Aware Raycast Splitting.	41
	Padding.	42
3.2.1.3	Data Augmentation Strategies	42
3.2.1.3.1	Permutations	42
3.2.1.3.2	Operations	43
3.2.2	Architecture Design	44
3.2.2.1	Design Considerations	44
3.2.2.2	Proposed Architecture Components	46
3.2.2.2.1	Training Strategy and Optimization	49
3.2.3	Curriculum	49
3.2.3.1	Assignment	50
3.2.3.2	Loss Function	50
3.2.4	Evaluation Procedure	51
3.2.4.1	Evaluation Metrics	51
3.2.4.2	Benchmarking	52
3.2.4.3	Generalization Test	52
3.2.4.4	Evaluation on Edge Device	52
3.3	Research Workflow	53
CHAPTER IV	Hasil dan Pembahasan	54

4.1	Pembahasan Tujuan 1 dengan Hasil Penelitian 1 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)	54
4.2	Pembahasan Tujuan 1 dengan Hasil Penelitian 2 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)	54
4.3	Pembahasan Tujuan 2 dengan Hasil Penelitian 3 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)	54
4.4	Perbandingan Hasil Penelitian dengan Hasil Terdahulu	55
CHAPTER V Tambahan (Opsional)		56
CHAPTER VI Kesimpulan dan Saran		57
6.1	Kesimpulan	57
6.2	Saran	57
REFERENCES		58
LAMPIRAN		L-1
L.1	Isi Lampiran	L-1
L.2	Panduan Latex	L-2
L.2.1	Syntax Dasar	L-2
L.2.1.1	Penggunaan Sitasi	L-2
L.2.1.2	Penulisan Gambar	L-2
L.2.1.3	Penulisan Tabel	L-2
L.2.1.4	Penulisan formula	L-2
L.2.1.5	Contoh list	L-3
L.2.2	Blok Beda Halaman	L-3
L.2.2.1	Membuat algoritma terpisah	L-3
L.2.2.2	Membuat tabel terpisah	L-3
L.2.2.3	Menulis formula terpisah halaman	L-4
L.3	Format Penulisan Referensi	L-6
L.3.1	Book	L-6
L.3.2	Handbook	L-8
L.4	Contoh Source Code	L-10
L.4.1	Sample algorithm	L-10
L.4.2	Sample Python code	L-11
L.4.3	Sample Matlab code	L-12

LIST OF TABLES

Table 2.1	Comparison of prior nuclei detection and segmentation methods. The Fully Convolutional column indicates whether the method uses a pure FCN backbone (no transformer or attention-based components).	6
Table 3.1	Summary of datasets used in this study.....	31
Table 3.2	Precomputed ray permutation indices for each geometric transform.	43
Table 4.1	Evaluation Against LSP-DETR.....	54
Table L.1	Tabel ini	L-2
Table L.2	Contoh tabel panjang	L-4

LIST OF FIGURES

Figure 2.1	Illustration of convex and star-shaped polygons. (a) A convex polygon: the segment $\overline{q_1q_2}$ connecting any two interior points lies entirely within the polygon. (b) A star-shaped polygon that is not convex: the kernel point z (near the center) sees all boundary vertices (dashed green), yet the segment $\overline{q_1q_2}$ connecting two interior points exits the polygon, demonstrating that the interior is not a convex set.	24
Figure 3.2	High-level Pipeline Design	32
Figure 3.3	A star-convex polygon with centroid c , polar grid, and 12 radial rays with angular spacing θ_i	32
Figure 3.4	RayCast representation of a cell polygon. From the area-weighted centroid P , $R = 32$ rays are cast at uniformly spaced angles. Each distance d_r records the nearest boundary intersection along ray r	33
Figure 3.5	UML class diagram of the modular ingestion pipeline. <code>BaseDataIngestor</code> defines the shared infrastructure; concrete ingestors implement format-specific <code>process_item</code> logic; all converge on the unified raycast annotation format via <code>RayCastGPU</code>	37
Figure 3.6	Population-level stain vector estimation. A stratified 20% subset of tiles is sampled per dataset and tissue type; the Macenko method estimates per-tile stain matrices and maximum concentrations, which are aggregated into the population profile $(\overline{M}, c_{\max}^*)$	40
Figure 3.7	Annotation-aware raycast splitting. (a) The ROI is divided into overlapping tiles. Only cells whose centroids fall inside a given tile are assigned to it (centroid-owns-cell policy). (b) For each assigned cell, rays extending beyond the tile boundary are clipped to the nearest wall.	42
Figure 3.8	RayCastED model architecture	44
Figure 3.9	ResoConv	46
Figure 3.10	RayCastDetect	47
Figure 3.11	Split Classification Head	48
Figure 3.12	Regression Head with RayCast Refinement	48
Figure 3.13	Evaluation Procedure Diagram	51
Figure 3.14	Research workflow diagram illustrating the iterative development process of the RayCastED model, from literature review and data preparation through model development, training, evaluation, and deployment.	53
Figure L.1	Contoh gambar.....	L-2

NOMENCLATURE AND ABBREVIATION

[SAMPLE]

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
$\mathbf{w}$	=	vektor bobot
$\mathbf{x}$	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Intisari ditulis menggunakan bahasa Indonesia dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Intisari sekurang-kurangnya berisi tentang latar belakang dan tujuan penelitian, metodologi yang digunakan, hasil penelitian, kesimpulan dan implikasi, dan Kata kunci yang berhubungan dengan penelitian.

Kata Kunci ditulis maksimal 5 kata yang paling berhubungan dengan isi skripsi. Silakan mengacu pada ACM / IEEE *Computing classification* jika Anda adalah mahasiswa Sarjana TI <http://www.acm.org/about/class/> atau mengacu kepada IEEE keywords http://www.ieee.org/documents/taxonomy_v101.pdf jika Anda berasal dari Prodi Sarjana TE.

Kata kunci : Kata kunci 1, Kata kunci 2, Kata kunci 3, Kata kunci 4, Kata kunci 5

Contoh Abstrak Teknik Elektro:

"Penelitian ini bertujuan untuk mengembangkan sistem pengendalian suhu ruangan dengan menggunakan microcontroller. Metodologi yang digunakan adalah desain sirkuit, implementasi sistem pengendalian, dan pengujian performa. Hasil penelitian menunjukkan bahwa sistem pengendalian suhu ruangan yang dikembangkan mampu mengendalikan suhu ruangan dengan akurasi sebesar $\pm 0,5^{\circ}\text{C}$. Kesimpulan dari penelitian ini adalah sistem pengendalian suhu ruangan yang dikembangkan efektif dan efisien.

Kata kunci: microcontroller, sistem pengendalian suhu, akurasi."

Contoh Abstrak Teknik Biomedis:

"Penelitian ini bertujuan untuk mengevaluasi keefektifan prototipe alat pemantau denyut jantung berbasis elektrokardiogram (ECG) untuk pasien jantung. Metodologi yang digunakan meliputi desain dan pembuatan prototipe, pengujian dengan pasien, dan analisis data. Hasil penelitian menunjukkan bahwa prototipe alat pemantau denyut jantung berbasis ECG memiliki akurasi yang baik dan mampu memantau denyut jantung pasien secara efektif. Kesimpulan dari penelitian ini adalah prototipe alat pemantau denyut jantung berbasis ECG merupakan solusi yang efektif dan efisien untuk memantau pasien jantung.

Kata kunci: elektrokardiogram, alat pemantau denyut jantung, akurasi."

Contoh Abstrak Teknologi Informasi:

"Penelitian ini bertujuan untuk mengevaluasi keamanan dan privasi pengguna aplikasi media sosial terpopuler. Metodologi yang digunakan meliputi analisis kebijakan privasi dan pengaturan keamanan, pengujian penetrasi, dan survei pengguna. Hasil penelitian menunjukkan bahwa beberapa aplikasi media sosial memiliki kebijakan privasi yang kurang jelas dan rendahnya tingkat keamanan. Kesimpulan dari penelitian ini adalah pentingnya meningkatkan kebijakan privasi dan tingkat keamanan pada aplikasi media sosial untuk melindungi privasi dan data pengguna.

Kata kunci: media sosial, keamanan, privasi, pengguna."

ABSTRACT

Abstract ditulis italic (miring) menggunakan bahasa Inggris dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Abstract adalah versi Bahasa Inggris dari intisari. Abstract dapat ditulis dalam beberapa paragraf. Baris pertama paragraph harus menjorok ke dalam sekitar 1 cm. Tidak disarankan menggunakan mesin penerjemah melainkan tulis ulang.

Keywords : Keyword 1, Keyword 2, Keyword 3, Keyword 4, Keyword 5

CHAPTER I

INTRODUCTION

1.1 Background

Nuclei segmentation is a critical first step in the analysis of histopathological images from Whole Slide Images (WSIs), as it underpins many downstream processes such as tumor grading, cellular composition analysis, and biomarker quantification [1]. In recent years, deep learning (DL) has become the staple approach for performing nuclei segmentation, driven by its ability to learn hierarchical features directly from data [1, 2]. Despite these advances, one of the principal difficulties that DL models continue to face is the presence of clumped nuclei, which results in overlapping and touching instances that are difficult to separate accurately.

Instance segmentation has emerged as the dominant paradigm for addressing this challenge, owing to its ability to classify individual objects as unique instances [3]. Among instance segmentation approaches, mask-based methods are the preferred choice in the literature [4]. However, mask-based instance segmentation carries notable drawbacks. First, these methods are computationally heavy, typically relying on two-stage architectures or elaborate post-processing pipelines [5]. Second, mask-based representations enforce pixel exclusivity: each pixel can belong to only one segment, which distorts morphology at contact boundaries where cells overlap [6].

An alternative line of work is contour-based instance segmentation, which represents objects through their boundary geometry rather than dense pixel masks. Contour-based methods offer two key advantages: they are inherently lighter, often employing single-stage architectures [5], and they can incorporate geometric priors into the representation prior to training [7]. Nevertheless, existing contour-based approaches remain dependent on non-maximum suppression (NMS) as a post-processing step, which adds computational overhead and risks removing valid predictions when instances are in close proximity [8]. Although vision transformer (ViT) based methods can eliminate the NMS dependency, they introduce substantially higher computational cost that limits their practicality.

This research proposes, **RayCastED**, a deep learning model that bridges the gap between these paradigms by combining three desirable properties. *Geometry-awareness*: the model employs ray-prediction based on star-convex polygons to capture true cellular boundaries, enabling precise area and morphological measurements without classifying every pixel. *Lightweight architecture*: the FCN backbone keeps computational overhead low, making the model suitable for edge deployment via TensorRT and ONNX Runtime. *Truly end-to-end*: the architecture completely eliminates the need for NMS or other

post-processing, allowing it to natively and accurately resolve dense, highly overlapping cellular scenarios while reducing inference overhead.

1.2 Problem Statement

Based on the background discussed above, the following problems are identified:

1. Touching or overlapping nuclei introduce computational overhead.
Existing methods address clumped nuclei either through two-stage architectures, heavy post-processing pipelines such as NMS, or computationally expensive vision transformer (ViT) backbones. Each of these strategies imposes a significant computational burden that limits scalability to large WSIs.
2. State-of-the-art methods are not suitable for edge deployment
Current leading approaches to nuclei segmentation and detection are designed with computational assumptions that exceed the capabilities of resource-constrained edge devices. Deploying these models in clinical settings requires hardware that is often unavailable at the point of care.

1.3 Objectives

To address the identified problems, this thesis has the following objectives:

1. Develop **RayCastED**, a nuclei detection model that:
 - represents cells through star-convex ray-predictions that capture true geometric boundaries without dense pixel-wise classification, addressing the pixel exclusivity limitation of mask-based methods;
 - employs a fully convolutional network (FCN) backbone to maintain a lightweight profile, addressing the computational overhead introduced by two-stage architectures, NMS post-processing, and ViT backbones (Problem 1);
 - operates in a truly end-to-end manner without NMS or other post-processing, enabling accurate detection in dense, overlapping cellular regions; and
 - is designed for edge deployment through TensorRT and ONNX Runtime compatibility, making it suitable for resource-constrained clinical environments (Problem 2).
2. Evaluate the performance of RayCastED by comparing it against prior nuclei detection and segmentation models on standard benchmarks.
3. Evaluate the deployment feasibility of RayCastED on edge devices.

1.4 Research Scope

This research is bounded by the following scope limitations:

1. **Data characteristics:** The study uses only H&E stained histopathology images with approximately 0.25 MPP (microns per pixel) resolution from publicly available secondary datasets. The datasets encompass multiple tissue types/organs.
2. **Computational constraints:** Model training is performed on RTX 5070Ti hardware. Baseline comparisons are limited to models compatible with this hardware configuration.
3. **Validation approach:** The research employs computational validation using standard metrics. Pathologist or expert validation is not included in this study.
4. **Technical boundaries:** There are no limitations on nuclei types, sizes, or detection scope. No strict constraints are imposed on input patch sizes or memory usage during model training and inference.

1.5 Research Novelty

The novelty of this research lies in the development of RayCastED, which proposes a deep learning architecture combining three elements: (1) a star-convex polygon representation that captures cell boundary geometry without dense pixel-wise classification, (2) a fully convolutional network backbone that maintains a lightweight computational profile suitable for edge deployment, and (3) a truly end-to-end design that eliminates NMS dependency entirely. This unique combination bridges the gap between the morphological accuracy of contour-based segmentation and the lightweight efficiency of single-shot detection for nuclei analysis in histopathology images.

1.6 Research Contributions

The contributions of this research are as follows:

1. **RayCastED model implementation:** A complete implementation of the proposed star-convex polygon-based detection model with raycast representation and end-to-end training, built on a fully convolutional backbone.
2. **Performance benchmarks on standard datasets:** Comprehensive evaluation of RayCastED on publicly available histopathology datasets, comparing performance against baseline models including mask-based instance segmentation, contour-based methods, and transformer-based approaches.
3. **Edge deployment validation:** Validation of RayCastED’s deployment capabilities on edge devices through TensorRT and ONNX Runtime conversion, demonstrating practical feasibility for resource-constrained clinical environments.

4. **Systematic comparison against baselines:** Quantitative comparison with prior nuclei detection and segmentation methods to measure improvements in boundary accuracy, counting precision in dense regions, and computational efficiency.
5. **Reproducible codebase:** A publicly available, well-documented implementation that enables reproducibility and provides a foundation for future research in polygon-based detection for computational pathology.

1.7 Presentation Structure

Sistematika penulisan berisi pembahasan apa yang akan ditulis di setiap bab. Sistematika pada umumnya berupa paragraf yang setiap paragraf mencerminkan bahasan setiap Bab. Bagian ini akan ditulis setelah seluruh bab dalam skripsi ini selesai disusun.

CHAPTER II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Literature Review

This section reviews prior work on nuclei instance segmentation in histopathology images, focusing on the methods most relevant to the proposed RayCastED model.

2.1.1 StarDist

StarDist [8] introduced the star-convex polygon representation for cell detection, originally developed for fluorescence microscopy and later adapted to histopathology images [9]. StarDist predicts a set of radial distances from each cell centroid to its boundary at fixed angular intervals, producing a star-convex polygon that preserves boundary geometry. However, StarDist relies on a two-stage pipeline: it first generates a dense probability map and then applies non-maximum suppression (NMS) to filter overlapping predictions, which risks undercounting in densely clumped regions and introduces computational overhead from post-processing.

2.1.2 CPP-Net

CPP-Net [10] proposed a context-aware polygon proposal network for nuclei instance segmentation. Rather than regressing radial distances from a fixed grid, CPP-Net generates polygon proposals by learning to predict polygon vertices directly, using a context-aware module that aggregates multi-scale features to refine proposal quality. While CPP-Net demonstrates that polygon-based representations can achieve high segmentation accuracy, it operates within a dense prediction framework and relies on NMS-based post-processing to remove duplicate proposals, inheriting the same NMS dependency as StarDist.

2.1.3 HoVer-NeXt

HoVer-NeXt [11] extended the HoVer-Net [12] framework for nuclei segmentation and classification, optimizing it for computational efficiency in large-scale histopathology applications. HoVer-Net predicts horizontal and vertical distance maps from each pixel to the centroids of neighboring cells, which are then used to separate touching or overlapping instances through post-processing. HoVer-NeXt improves upon this pipeline with architectural refinements that reduce inference time while maintaining segmentation quality. However, the method remains fundamentally a dense segmentation approach that classifies every pixel and requires computationally expensive post-processing (watershed algorithms and morphological operations) to extract individual instance masks.

2.1.4 CellPose

CellPose [13] introduced a generalist framework for cellular segmentation that predicts a gradient flow field pointing from each pixel toward the nearest cell centroid, combined with a cell probability map. Trained on a diverse dataset of over 70,000 segmented objects across multiple image types, CellPose demonstrates strong cross-domain generalization without requiring model retraining. However, CellPose operates as a dense segmentation model that classifies every pixel, making it computationally expensive for whole-slide analysis. A subsequent extension, CellPose-SAM [14], combined CellPose with the Segment Anything Model (SAM) to achieve superhuman generalization, though the computational profile remains unsuitable for resource-constrained deployment.

2.1.5 LSP-DETR

LSP-DETR [15] represents the most direct precursor to this work. It introduced a fully end-to-end framework for nuclei segmentation that represents nuclei as star-convex polygons within a DETR-style transformer architecture. By using a learned bipartite assignment mechanism instead of NMS, LSP-DETR eliminates the undercounting problem in clumped regions. The authors also proposed a novel radial distance loss that allows overlapping nuclei segmentation to emerge naturally. LSP-DETR demonstrated strong generalization across tissues and was over five times faster than the next-fastest leading method. However, its transformer-based backbone imposes quadratic complexity, making it computationally demanding for edge deployment. RayCastED is heavily inspired by LSP-DETR’s use of star-convex polygon predictions within an end-to-end, NMS-free framework, but is designed from the ground up with a fully convolutional architecture to maintain a lightweight computational profile suitable for resource-constrained environments.

Table 2.1 summarizes the key characteristics of the reviewed methods, highlighting how each method addresses (or fails to address) the core challenges identified in the background.

Table 2.1. Comparison of prior nuclei detection and segmentation methods. The **Fully Convolutional** column indicates whether the method uses a pure FCN backbone (no transformer or attention-based components).

Method	Approach	End-to-End	Fully Convolutional	Post-processing
StarDist [8, 9]	Detection	×	✓	NMS
CPP-Net [10]	Instance Seg.	×	✓	NMS
HoVer-NeXt [11, 12]	Instance Seg.	×	✓	Watershed + Morphological
CellPose [13, 14]	Instance Seg.	×	✓	NMS
LSP-DETR [15]	Detection	✓	×	None

As Table 2.1 shows, no existing method simultaneously achieves all three prop-

erties: end-to-end detection, polygon-based boundary representation, and a fully convolutional architecture. StarDist, CPP-Net, HoVer-NeXt, and CellPose all rely on FCN backbones but require handcrafted post-processing. LSP-DETR is end-to-end through bipartite matching but relies on a transformer backbone. RayCastED is designed to fill this gap by combining the end-to-end bipartite assignment of LSP-DETR with a purely FCN-based architecture.

2.2 Theoretical Basis

2.2.1 Histopathological Foundations

Histopathology is the study of tissue disease through microscopic examination of tissue samples. In clinical practice, tissue specimens are collected through biopsy or surgical resection and processed for microscopic analysis. The tissue undergoes fixation (typically in formalin) to preserve cellular structures, followed by embedding in paraffin wax, sectioning into thin slices, and mounting on glass slides for staining and visualization.

The cell nucleus is a membrane-bound organelle containing genetic material and serves as the primary structure of interest in histopathological analysis. Nuclear morphology, including size, shape, texture, and spatial distribution, provides critical diagnostic information. In cancer diagnosis, malignant cells often exhibit nuclear atypia: enlarged nuclei, irregular shapes, hyperchromasia (increased staining intensity), and abnormal mitotic figures.

2.2.1.1 Nuclear Morphology and Shape Characteristics

The shape of cell nuclei varies considerably across tissue types and pathological states, but the majority of nuclei encountered in routine H&E-stained histopathology sections exhibit a defining geometric property: they are **approximately convex**. Healthy epithelial nuclei are typically round or slightly oval, with smooth contours and minimal indentations. Lymphocytes are among the most regular, appearing as compact circles with diameters of approximately 5–8 μm . Fibroblast and endothelial nuclei tend to be elongated (spindle-shaped), but remain convex with no concavities. Even in malignant tissue, where nuclear pleomorphism introduces greater variability in size and eccentricity, most nuclei retain a fundamentally convex silhouette. Concavities do arise in practice, most commonly at the boundary between two overlapping nuclei where the cells appear to “press into” one another, but these are artifacts of the 2D projection rather than intrinsic features of the nuclear membrane itself.

This near-ubiquitous convexity has an important implication for automated representation. Because concave shapes are rarely encountered in their undistorted form, nuclear boundaries can be fully described by a centroid and a set of radial distances measured from that centroid to the boundary at regular angular intervals. This radial

representation captures the complete boundary geometry with a compact set of parameters, avoiding both the redundancy of dense per-pixel segmentation and the information loss of axis-aligned bounding boxes. The few cases where concavities do appear, primarily at overlap boundaries, are artifacts of the 2D projection and can be addressed by representing each overlapping nucleus independently rather than forcing a single non-overlapping mask.

2.2.1.2 The Overlapping Nuclei Problem

A fundamental challenge in nuclei analysis is that nuclei in tissue sections are densely packed and frequently overlap, forming clumped clusters where individual boundaries are difficult to distinguish even for expert pathologists. This overlapping occurs because tissue sections are typically cut at a thickness of 3–5 μm , which is comparable to or smaller than the diameter of many nuclei (typically 5–10 μm). As a result, multiple nuclei at different depths within the section are projected onto the same 2D image plane, creating the appearance of clumped or touching nuclei. This is not merely an artifact of imaging resolution but an inherent property of projecting 3D tissue architecture onto a 2D plane, motivating the development of automated methods capable of both accurately delineating individual nuclear boundaries and correctly separating touching or overlapping nuclei.

2.2.1.3 Area-based Measurements

Tumor-infiltrating lymphocytes (TILs) are immune cells, primarily lymphocytes and plasma cells, that have migrated into the tumor microenvironment. The presence and density of TILs reflect the host immune response against the tumor and have been established as a prognostic biomarker in multiple cancer types. Higher TILs levels are generally associated with improved outcomes, particularly in triple-negative and HER2-positive breast cancers, where they predict better response to chemotherapy and longer survival.

The International TILs Working Group, convened by Salgado et al. [16], established standardized guidelines for TILs assessment in clinical practice. The recommended metric is **stromal TILs (sTILs)**, defined as the proportion of stromal area occupied by mononuclear inflammatory cells within the invasive tumor border:

$$\text{sTILs (\%)} = \frac{\text{Area of mononuclear inflammatory cells in stroma}}{\text{Total intratumoral stromal area}} \times 100 \quad (2-1)$$

Only lymphocytes and plasma cells are included; polymorphonuclear leukocytes are excluded. Areas of necrosis, crush artifact, fibrosis, and hyalinization at the tumor

border are also excluded from the assessment. The score is reported as a single average percentage per tumor on H&E-stained full-tissue sections.

Clinically, tumors with sTILs $\geq 50\%$ are classified as lymphocyte-predominant breast cancer (LPBC), indicating a strong immune response. Additional commonly used thresholds include sTILs $< 10\%$ (TILs-low) and sTILs $\geq 30\%$ (TILs-high in many triple-negative breast cancer studies).

This scoring methodology is fundamentally an *area-based* measurement: it requires accurately delineating the boundary of each lymphocyte to compute its area contribution, not merely detecting its presence or counting its centroid. This distinction has direct implications for the design of automated nuclei analysis systems. Object detection models that output only bounding boxes or centroids are insufficient for sTILs scoring because they discard boundary geometry. Accurate sTILs computation requires methods that can both detect individual cells and delineate their boundaries, particularly in densely clumped regions where cell boundaries are ambiguous.

2.2.1.4 Cell Count-based Measurements

An alternative approach to quantifying immune infiltration is through cell count-based measurements, which estimate the number of immune cells per unit area rather than computing their proportional area. The most prominent example is the **Immunoscore**, developed and validated by Pagès et al. [17] for colorectal cancer. The Immunoscore quantifies the density of CD3⁺ and CD8⁺ T lymphocytes within two histological regions: the **tumor core** (CT) and the **invasive margin** (IM). This yields four density values, expressed as the number of positive cells per square millimeter:

$$\text{IS} = \{d_{\text{CD3}}^{\text{CT}}, d_{\text{CD3}}^{\text{IM}}, d_{\text{CD8}}^{\text{CT}}, d_{\text{CD8}}^{\text{IM}}\} \quad (2-2)$$

where d_m^r denotes the density of marker $m \in \{\text{CD3}, \text{CD8}\}$ in region $r \in \{\text{CT}, \text{IM}\}$. Each density value is classified as high (1) or low (0) based on a predefined percentile threshold determined from a reference patient cohort. The four binary values are then summed to produce the final Immunoscore:

$$\text{Immunoscore} = \sum_{m \in \{\text{CD3}, \text{CD8}\}} \sum_{r \in \{\text{CT}, \text{IM}\}} \mathbb{I}[d_m^r \geq \tau_m^r] \quad (2-3)$$

where τ_m^r is the density threshold for marker m in region r and $\mathbb{I}[\cdot]$ is the indicator function. The resulting score ranges from I0 (all four densities low) to I4 (all four densities high), with higher scores indicating stronger immune infiltration and consistently associated with improved patient survival across multiple validation cohorts.

Unlike area-based TILs scoring, count-based measurements do not require accurate boundary delineation of each individual cell. A detection model that correctly identifies cell centroids is sufficient to count cells per unit area. However, count-based methods typically require immunohistochemical (IHC) staining for specific markers (e.g., CD3, CD8) rather than standard H&E staining, which limits their applicability in resource-constrained settings where IHC is not routinely available. Furthermore, the area-based sTILs methodology has been more widely adopted across cancer types, particularly in breast cancer, making it the more broadly relevant measurement for automated analysis systems.

2.2.2 Digital Pathology Imaging

2.2.2.1 Tissue Slicing and Scanning

The digitization of histopathology slides produces whole-slide images (WSIs) through specialized scanners that capture high-resolution images of the entire tissue section. Modern slide scanners, such as those from Hamamatsu, Leica, and 3DHistech, use line-sensor or area-sensor cameras mounted on motorized stages to systematically capture overlapping tiles that are subsequently stitched into a single large image.

WSIs are typically stored in multi-resolution pyramidal formats (e.g., Aperio SVS, Hamamatsu NDP, or generic TIFF), where each level provides a different magnification. The spatial resolution of a WSI is specified in microns per pixel (MPP), which depends on the objective lens magnification used during scanning. Common resolutions range from approximately 0.25 MPP (equivalent to $40\times$ magnification) to 0.5 MPP ($20\times$). At 0.25 MPP, a typical biopsy slide can produce an image exceeding $50,000 \times 50,000$ pixels, necessitating patch-based processing strategies for computational analysis.

2.2.2.2 Histopathological Slides Staining

Tissue staining is essential because most biological tissues are nearly transparent under standard brightfield microscopy. Stains introduce contrast by selectively binding to tissue components through chemical interactions. The most widely used staining protocol in histopathology is Hematoxylin and Eosin (H&E) staining.

Hematoxylin is a basic (positively charged) dye that binds to acidic structures such as DNA and RNA, coloring nuclei dark blue or purple. Eosin is an acidic (negatively charged) dye that binds to basic structures such as cytoplasmic proteins and extracellular matrix, coloring them pink. The combination produces the characteristic appearance of H&E-stained tissue: blue-purple nuclei against a pink cytoplasmic and stromal background.

Beyond H&E, several specialized stains are used in clinical practice: Periodic

Acid-Schiff (PAS) for basement membranes and glycogen, Masson's Trichrome for collagen fibers, and immunohistochemical (IHC) stains for specific protein markers. Each stain produces distinct color characteristics that can be mathematically decomposed for computational analysis.

2.2.2.3 Beer-Lambert Law and Optical Density

The interaction of light with stained tissue follows the Beer-Lambert law, which describes the relationship between light absorption and the concentration of absorbing material. For a stained tissue sample, the transmitted intensity I at a given wavelength is related to the incident intensity I_0 by:

$$I = I_0 \cdot e^{-\alpha \cdot c \cdot d} \quad (2-4)$$

where α is the molar absorptivity (extinction coefficient), c is the concentration of the stain, and d is the path length through the tissue. The optical density (OD) is defined as:

$$\text{OD} = -\log_{10} \left(\frac{I}{I_0} \right) = \alpha \cdot c \cdot d \cdot \log_{10}(e) \quad (2-5)$$

In the context of digital pathology, the RGB pixel values of a scanned image are proportional to transmitted light intensity. Converting from RGB to optical density space is essential because OD is linearly additive: if multiple stains are present, the total OD at each pixel equals the sum of the OD contributions from each individual stain. This linearity property underpins color deconvolution methods.

2.2.2.4 Stain Vectors

A stain vector represents the characteristic color signature of a specific stain in optical density space. For an RGB image, each stain is described by a 3-element unit vector $\mathbf{s} = [s_R, s_G, s_B]^T$ in OD space, where each component indicates the stain's relative absorption at the red, green, and blue channels.

For H&E staining, the two stain vectors are:

- **Hematoxylin vector $\mathbf{s}_H$:** characterizes the absorption pattern of hematoxylin, which strongly absorbs in the blue-green range, producing its characteristic dark blue appearance.
- **Eosin vector $\mathbf{s}_E$:** characterizes the absorption pattern of eosin, which absorbs primarily in the green range, producing its pink appearance.

When two stains are present, the stain matrix S is formed by concatenating the

stain vectors as columns: $S = [s_H \ s_E]$. For three-stain protocols, a third column is added.

2.2.2.5 Stain Vector Estimation Methods

Since stain appearance varies across scanners, laboratories, and tissue types, the stain vectors must be estimated from each individual image or dataset. Several methods exist for automatic stain vector estimation:

Macenko's method [18] estimates stain vectors by analyzing the angular distribution of pixel OD values through Singular Value Decomposition (SVD). The procedure operates as follows.

Given an RGB image with N pixels, each pixel $\mathbf{p}_i = [R_i, G_i, B_i]^T$ is first converted to optical density space:

$$\mathbf{o}_i = -\log_{10} \left(\frac{\mathbf{p}_i}{255} + \epsilon \right) \quad (2-6)$$

where ϵ is a small constant to avoid logarithm of zero. Pixels whose OD magnitude $\|\mathbf{o}_i\|$ falls below a threshold t_{OD} are discarded, as they correspond to near-white background regions with negligible stain absorption. The threshold is typically set at the 1st percentile of the OD magnitude distribution.

For the remaining M pixels, a matrix $\hat{O} \in \mathbb{R}^{3 \times M}$ is formed by arranging the OD vectors as columns. SVD is then applied:

$$\hat{O} = U \Sigma V^T \quad (2-7)$$

where $U \in \mathbb{R}^{3 \times 3}$ contains the left singular vectors. The two principal singular vectors $\mathbf{u}_1$ and $\mathbf{u}_2$ (the first two columns of U) span the plane in which the stain vectors lie. Each OD vector is projected onto this plane:

$$\mathbf{p}'_i = \begin{bmatrix} \mathbf{u}_1^T \mathbf{o}_i \\ \mathbf{u}_2^T \mathbf{o}_i \end{bmatrix} \quad (2-8)$$

and converted to angular coordinates $\theta_i = \arctan 2(p'_{i,2}, p'_{i,1})$. The stain directions are determined by finding the angles $\theta_{\min}$ and $\theta_{\max}$ that enclose the 1st and 99th percentiles of the angular distribution, corresponding to the extreme concentrations of each stain.

The raw stain vectors are then reconstructed in OD space:

$$\mathbf{v}_1 = \mathbf{u}_1 \cos \theta_{\min} + \mathbf{u}_2 \sin \theta_{\min}, \quad \mathbf{v}_2 = \mathbf{u}_1 \cos \theta_{\max} + \mathbf{u}_2 \sin \theta_{\max} \quad (2-9)$$

Since SVD produces vectors with arbitrary sign, the vectors are flipped such that all components are non-negative (as OD values are inherently non-negative). Finally, the

vectors are normalized to unit length: $\mathbf{s}_j = \mathbf{v}_j / \|\mathbf{v}_j\|$.

To assign identities, the angle each normalized vector makes with a predefined reference direction in OD space (typically corresponding to a hematoxylin-like hue) is computed. The vector closer to the hematoxylin reference is labeled $\mathbf{s}_H$, and the other is labeled $\mathbf{s}_E$.

A residual stain vector is then computed via the cross product of the two estimated stain vectors:

$$\mathbf{s}_R = \mathbf{s}_H \times \mathbf{s}_E \quad (2-10)$$

This third vector is orthogonal to both $\mathbf{s}_H$ and $\mathbf{s}_E$ and captures the residual OD component not explained by the two primary stains, arising from background variation, optical noise, or imaging artifacts. It is included as the third column of the stain matrix to form a complete basis $S = [\mathbf{s}_H \ \mathbf{s}_E \ \mathbf{s}_R] \in \mathbb{R}^{3 \times 3}$, which is required for the matrix inversion in color deconvolution (Section??). Without this third vector, the stain matrix would be rank-deficient and non-invertible.

Macenko's method is widely adopted due to its computational simplicity and robustness, as the SVD-based projection efficiently captures the dominant stain directions without iterative optimization.

2.2.2.6 Color Deconvolution

Color deconvolution is the process of separating an H&E-stained image into its individual stain channels. Given the stain matrix S and the OD image $\mathbf{v}$ (a pixel vector in OD space), the stain concentrations $\mathbf{c}$ for each pixel are obtained by solving the linear system:

$$\mathbf{v} = S \cdot \mathbf{c} \quad (2-11)$$

Since S is a 3×2 matrix (for H&E) or 3×3 (for three stains), the system can be solved using the pseudoinverse:

$$\mathbf{c} = S^{-1} \cdot \mathbf{v} \quad (2-12)$$

where S^{-1} denotes the pseudoinverse (or inverse, for square S). The resulting stain concentration maps $\mathbf{c}$ provide separate grayscale images representing the hematoxylin and eosin density at each pixel. The hematoxylin channel is particularly useful for nuclei detection algorithms, as it isolates nuclear staining from cytoplasmic and stromal background.

2.2.2.7 Stain Normalization

Stain normalization addresses the significant color variation that exists across WSIs captured from different scanners, hospitals, or staining batches. This variation, if uncorrected, degrades the performance of deep learning models trained on data from one source and applied to another.

The standard approach to stain normalization involves three steps: (1) estimate the stain matrix S_{source} from the source image using one of the methods described above, (2) compute the stain concentration map $\mathbf{c} = S_{\text{source}}^{-1} \cdot \mathbf{v}_{\text{source}}$, and (3) reconstruct the normalized image using a target stain matrix S_{target} : $\mathbf{v}_{\text{normalized}} = S_{\text{target}} \cdot \mathbf{c}$, then convert back from OD to RGB space.

This process preserves the structural content of the image while standardizing color appearance. Stain normalization is commonly applied as a preprocessing step before training or inference in computational pathology pipelines.

2.2.3 Segmentation Paradigms

2.2.3.1 Semantic Segmentation

Semantic segmentation assigns a class label to every pixel in an image, producing a dense classification map where each pixel is labeled as belonging to one of several classes. Formally, given an input image $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$, a semantic segmentation model produces a label map $\mathbf{Y} \in \{1, \dots, K\}^{H \times W}$ where K is the number of classes and each $Y_{i,j}$ indicates the class of pixel (i, j) .

The standard architecture for semantic segmentation is the fully convolutional network (FCN), which replaces the fully connected layers of a classification network with convolutional layers to produce spatially dense outputs. The encoder-decoder structure is common: an encoder backbone (e.g., ResNet, VGG) progressively downsamples the input to extract high-level features, while a decoder progressively upsamples these features to recover the original spatial resolution. Skip connections between corresponding encoder and decoder layers help preserve fine-grained spatial details that would otherwise be lost during downsampling.

Semantic segmentation is effective for macro-level area measurements, such as estimating the overall proportion of stromal tissue occupied by lymphocytes. However, it does not distinguish between individual instances of the same class: all nuclei receive the same label regardless of their spatial separation, making it impossible to count individual cells.

2.2.3.2 Instance Segmentation

Instance segmentation extends semantic segmentation by assigning a unique label to each individual object instance, producing both a class label and an instance identity for every pixel. Given an input image, the output is a label map where each pixel is assigned a pair (c, k) indicating that it belongs to instance k of class c .

Two dominant paradigms exist for instance segmentation. **Proposal-based methods** such as Mask R-CNN [19] operate in two stages: a region proposal network (RPN) generates candidate object bounding boxes, and a mask head then predicts a binary segmentation mask for each proposal. While accurate, the two-stage pipeline introduces significant computational overhead, particularly when the number of instances is large.

Dense prediction methods such as U-Net [20] and HoVer-Net [12] predict instance information directly from dense feature maps. U-Net produces pixel-level embeddings where each pixel is mapped to a high-dimensional space, and instances are separated by clustering pixels based on embedding distance. HoVer-Net predicts horizontal and vertical distance maps from each pixel to the centroids of neighboring cells, using these maps to separate touching or overlapping instances. While these methods can achieve high accuracy, they require computationally expensive post-processing (e.g., watershed algorithms, clustering, or morphological operations) to convert dense predictions into individual instance masks.

2.2.3.3 Limitations of Segmentation for Nuclei Analysis

Segmentation-based approaches face two fundamental limitations when applied to nuclei analysis in histopathology images.

The **overlap problem** arises from the mutually exclusive pixel assignment inherent in segmentation. In a two-dimensional image, each pixel can only belong to one instance. When nuclei overlap in 3D tissue and are projected onto a 2D plane, their boundaries become entangled: a single pixel at the boundary between two overlapping nuclei cannot simultaneously represent both cells. Segmentation models resolve this ambiguity by drawing an artificial boundary between the two cells, which distorts the true morphology of both. This distortion is particularly problematic for nuclei analysis, where accurate boundary delineation directly affects area-based measurements such as TILs scoring.

The **computational bottleneck** stems from the dense prediction paradigm: segmentation models must classify every pixel in the image, including the vast majority of background pixels that carry no useful information. For a typical histopathology patch of 256×256 pixels, this means producing 65,536 individual predictions, even when only a small fraction correspond to actual nuclei. This per-pixel cost becomes prohibitive when processing whole-slide images, which may contain billions of pixels. The post-processing

required to extract individual instances from dense predictions (clustering, watershed, morphological operations) adds further computational overhead.

2.2.4 Detection Paradigms

Object detection models operate by dividing the input image into a spatial grid and predicting a fixed set of parameters (typically a bounding box and a class label) at each spatial location. Unlike segmentation, which classifies every pixel, detection produces a sparse set of structured outputs that describe the position and extent of individual objects. This grid-based prediction framework forms the foundation of modern single-stage detectors such as YOLO [21] and two-stage detectors such as Faster R-CNN [22], and is the paradigm adopted in this work.

2.2.4.1 Anchor Grid

In grid-based detection frameworks, predictions are organized around an **anchor grid**: a regular lattice of reference boxes (anchors) placed at each spatial location of a feature map. At each grid cell, multiple anchors of different scales and aspect ratios are defined to accommodate objects of varying sizes and shapes. Each anchor serves as a prior that the network refines into a final prediction by regressing coordinate offsets relative to the anchor dimensions and classifying the object category.

The anchor grid determines several key properties of the detector. It defines the maximum number of detections the model can produce: for a feature map of resolution $H \times W$ with A anchors per cell, the model generates up to $H \times W \times A$ predictions. Each grid cell is responsible for detecting objects whose centroids fall within its receptive field, creating a natural spatial correspondence between image regions and model outputs. This regular grid structure enables the detector to process objects at multiple scales simultaneously by operating on feature maps at different resolutions, with coarser grids detecting larger objects and finer grids detecting smaller objects.

2.2.4.2 Matching Mechanisms in Object Detection

A fundamental challenge in training detection models is establishing the correspondence between predicted outputs and ground-truth objects, a process known as **matching**. Given the anchor grid produces a large number of predictions per image, the matching mechanism determines which predictions are assigned to which ground-truth objects during loss computation.

During training, predictions are matched to ground-truth objects using an assignment rule. The assignment is formulated by constructing a **cost matrix** (also called a score matrix, depending on whether lower or higher values indicate better matches) that quantifies the quality of each prediction–ground-truth pair. Given N ground-truth objects

and M predictions, the cost matrix $\mathbf{C} \in \mathbb{R}^{N \times M}$ is defined such that element $C_{i,j}$ measures the dissimilarity between ground-truth object i and prediction j . The most common choice for $C_{i,j}$ is the negative IoU:

$$C_{i,j} = 1 - \text{IoU}(g_i, p_j) \quad (2-13)$$

where g_i is the bounding box (or polygon) of ground-truth object i and p_j is the predicted bounding box (or polygon). Since IoU ranges from 0 to 1, the cost $C_{i,j}$ also ranges from 0 (perfect overlap) to 1 (no overlap). Alternatively, some frameworks use a score matrix $\mathbf{S} \in \mathbb{R}^{N \times M}$ where $S_{i,j} = \text{IoU}(g_i, p_j)$ and the goal is to maximize the total score rather than minimize the total cost. The two formulations are equivalent: minimizing $\sum C_{i,j}$ is the same as maximizing $\sum S_{i,j}$ when $C_{i,j} = 1 - S_{i,j}$.

In addition to localization cost, the cost matrix may incorporate a **classification cost** term that penalizes mismatched class predictions. For multi-class detection with K classes, the classification cost can be defined using the cross-entropy between the predicted class distribution and the ground-truth class:

$$C_{i,j}^{\text{cls}} = -\log \hat{p}_j(c_i) \quad (2-14)$$

where $\hat{p}_j(c_i)$ is the predicted probability of class c_i (the true class of ground-truth i) and the total cost combines both terms:

$$C_{i,j} = C_{i,j}^{\text{loc}} + \lambda_{\text{cls}} \cdot C_{i,j}^{\text{cls}} \quad (2-15)$$

with λ_{cls} controlling the relative weight of classification versus localization. This combined cost matrix encodes both spatial alignment and semantic correctness into a single assignment objective.

Once the cost matrix is constructed, an assignment algorithm operates on it to determine which predictions correspond to which ground-truth objects. The choice of assignment algorithm determines whether the matching is one-to-many (allowing multiple predictions per ground-truth object), many-to-one (allowing one prediction to be matched to multiple ground-truth objects), or one-to-one (bipartite). Traditional methods such as Faster R-CNN [22] use IoU-based thresholding on this matrix: a prediction is assigned to a ground-truth object if $C_{i,j}$ falls below a threshold. This assignment is often ambiguous when multiple predictions overlap with the same ground-truth object, requiring heuristics to resolve conflicts. Modern YOLO variants use more sophisticated strategies, such as task-aligned matching and dual-label assignment, which extend the cost matrix with additional alignment terms. These strategies are discussed in Section 2.2.8.

2.2.4.3 Feature Pyramid Networks

A single feature map at a fixed resolution presents a fundamental limitation for object detection: objects in natural images vary widely in scale, and features that are discriminative for large objects (derived from deep, low-resolution layers with large receptive fields) may be too coarse to localize small objects, while features suitable for small objects (from shallow, high-resolution layers) may lack the semantic context needed to recognize large objects. **Feature Pyramid Networks (FPN)** [23] address this by constructing a multi-scale feature pyramid from a single input image in a computationally efficient manner.

An FPN extends a standard convolutional backbone with a top-down pathway and lateral connections. The backbone computes a hierarchy of feature maps $\{C_2, C_3, C_4, C_5\}$ at progressively coarser resolutions (downsampled by factors of 4, 8, 16, and 32 relative to the input). The top-down pathway then upsamples the deepest feature map C_5 and merges it with the corresponding lateral feature map from the backbone (e.g., C_4) via element-wise addition after applying a 1×1 convolution to match channel dimensions. This process is repeated to produce the pyramid levels $\{P_2, P_3, P_4, P_5\}$:

$$P_\ell = \text{Conv}_{1 \times 1}(C_\ell) + \text{Upsample}(P_{\ell+1}) \quad (2-16)$$

Each pyramid level P_ℓ has the same spatial resolution as its corresponding backbone feature map C_ℓ but benefits from the semantic richness of higher-level features propagated downward through the top-down pathway.

In a detector, each pyramid level is assigned to objects within a specific scale range. Large objects are detected at coarser levels (P_5) where the large receptive field captures holistic shape, while small objects are detected at finer levels (P_2) where high spatial resolution preserves precise localization. This per-level assignment reduces ambiguity in the matching process: objects of clearly different scales are assigned to different pyramid levels, preventing them from competing for the same prediction slot at a single resolution.

However, FPN also contributes to duplicate predictions. When an object falls near the boundary between two assigned scale ranges, it may be detected at two adjacent pyramid levels simultaneously, producing two spatially overlapping predictions for the same object. These cross-level duplicates are a significant source of redundancy that downstream NMS must resolve.

2.2.4.4 Non-Maximum Suppression

Non-maximum suppression (NMS) is a post-processing step used by most detection frameworks to eliminate duplicate predictions. When multiple predictions overlap with

the same ground-truth object, NMS selects the prediction with the highest confidence score and suppresses (removes) all other predictions whose overlap with the selected one exceeds a predefined IoU threshold.

The standard NMS procedure operates as follows. Given a set of detections ranked by confidence score, the highest-confidence detection is selected and all remaining detections with IoU exceeding a threshold τ_{nms} are suppressed. This process repeats until no detections remain.

Formally, let $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ be a set of detections sorted in descending order of confidence s_i , each with bounding box B_i . The NMS-filtered output set $\mathcal{D}'$ is defined inductively:

$$\mathcal{D}' = \{d_i \in \mathcal{D} : \forall d_j \in \mathcal{D}' \text{ with } s_j > s_i, \text{IoU}(B_i, B_j) < \tau_{\text{nms}}\} \quad (2-17)$$

Alternatively, expressed as a greedy iteration that produces a maximal independent set with respect to the τ_{nms} -thresholded neighborhood graph: starting from $\mathcal{D}' = \emptyset$, for each $d_i \in \mathcal{D}$ in confidence order, if $\max_{d_j \in \mathcal{D}'} \text{IoU}(B_i, B_j) < \tau_{\text{nms}}$, then $\mathcal{D}' \leftarrow \mathcal{D}' \cup \{d_i\}$.

The IoU threshold τ_{nms} controls the trade-off between eliminating duplicates and preserving valid detections of closely spaced objects.

NMS introduces a fundamental tension in nuclei detection. In densely clumped regions, multiple nuclei overlap significantly, producing predictions that are spatially close and have high mutual IoU. A low NMS threshold aggressively suppresses these predictions, reducing duplicates but also removing valid detections of distinct cells. A high threshold preserves more predictions but allows duplicates to remain, inflating the cell count. Neither setting resolves the underlying problem: NMS cannot distinguish between two overlapping predictions of the same cell and two valid predictions of adjacent cells that happen to overlap.

2.2.4.5 Limitations of Detection for Nuclei Analysis

Detection models offer computational advantages over segmentation but face two critical shortcomings when applied to nuclei analysis.

The **boundary problem** is that bounding box representations discard cell morphology. A bounding box encloses the approximate spatial extent of a cell but does not capture its shape. For approximately circular nuclei, the difference between bounding box area and true cell area may be modest, but for irregularly shaped or overlapping nuclei, bounding boxes significantly overestimate cell size and destroy the geometric information necessary for area-based measurements such as TILs scoring. A detection model can identify where cells are but cannot determine how much area each cell occupies.

The **density problem** compounds the boundary problem in the very regions where accuracy matters most. In densely clumped areas, cells are closely spaced and their bounding boxes overlap heavily. NMS must then decide which overlapping boxes correspond to distinct cells and which are duplicate predictions of the same cell. Since this decision is based solely on geometric overlap and confidence scores, NMS systematically undercounts in clumped regions: valid detections of adjacent cells are suppressed because their boxes overlap, while in sparse regions where cells are well-separated, NMS has little effect. This creates a density-dependent counting bias where the model performs worst precisely in the regions most relevant to clinical assessment.

2.2.5 Convolutional Neural Networks

2.2.5.1 Convolution and Feature Extraction

A convolutional layer applies a set of learnable filters (kernels) to an input feature map, producing an output feature map where each element is a weighted sum of local input elements. For a 2D convolution with input $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and kernel $\mathbf{W} \in \mathbb{R}^{K \times K \times C_{\text{in}} \times C_{\text{out}}}$, the output at spatial position (i, j) for channel c is:

$$\mathbf{Y}_{i,j,c} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} \sum_{k=0}^{C_{\text{in}}-1} \mathbf{W}_{m,n,k,c} \cdot \mathbf{X}_{i+m,j+n,k} + b_c \quad (2-18)$$

where b_c is the bias term for output channel c . Each kernel slide across the input computes the same operation at every spatial position, exploiting the translation invariance of visual patterns. Stacking multiple convolutional layers with nonlinear activation functions (typically ReLU) enables the network to learn hierarchical feature representations: early layers capture low-level features such as edges and textures, while deeper layers capture higher-level structures such as cell shapes and tissue patterns.

2.2.5.2 Receptive Field and Kernel Size

The receptive field of a neuron in a convolutional network is the region of the input image that influences its output. For a single convolutional layer with kernel size K , each neuron has a receptive field of $K \times K$ pixels. Stacking L convolutional layers, each with kernel size K and stride 1, produces a receptive field of size $(K + (K - 1)(L - 1)) \times (K + (K - 1)(L - 1))$ at the deepest layer. Using stride $s > 1$ or pooling layers between convolutions further enlarges the effective receptive field.

The kernel size directly determines the model’s ability to capture spatial context. A 3×3 kernel sees only immediate neighbors, while a 7×7 kernel captures a larger local context. However, larger kernels increase the number of parameters quadratically: a 7×7 kernel has 5.4 times more parameters than a 3×3 kernel for the same number of channels.

Modern architectures typically use small kernels (3×3) stacked deeply to achieve large effective receptive fields while keeping the parameter count manageable.

For nuclei detection, a sufficiently large receptive field is critical because nuclei can vary significantly in size, and contextual information from surrounding tissue helps distinguish nuclei from artifacts and debris. Methods such as LKCell [24] demonstrate that using large convolution kernels (e.g., 7×7 or larger) with structural reparameterization can achieve large receptive fields with lower computational cost than stacking many small-kernel layers.

2.2.5.3 Depthwise Convolution

Depthwise convolution is a spatial decomposition of the standard convolution operation that significantly reduces computational cost. In a standard convolution with C_{in} input channels and C_{out} output channels using a $K \times K$ kernel, the number of multiplications per spatial position is $K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$. Depthwise convolution decomposes this into two steps:

1. **Depthwise step:** Each of the C_{in} input channels is convolved independently with its own $K \times K$ kernel, producing C_{in} intermediate feature maps. Cost: $K^2 \cdot C_{\text{in}}$ multiplications per position.
2. **Pointwise step:** A 1×1 convolution (pointwise convolution) then combines the C_{in} intermediate maps into C_{out} output channels. Cost: $C_{\text{in}} \cdot C_{\text{out}}$ multiplications per position.

The total cost is $K^2 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$ per position, compared to $K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$ for standard convolution. The reduction factor is approximately K^2 , making depthwise convolution particularly effective for larger kernel sizes. This efficiency makes depthwise separable convolutions a key building block in lightweight architectures, as demonstrated by MobileNet [25] and adopted in subsequent YOLO variants for efficient feature extraction.

2.2.6 Discrete Wavelet Transform

2.2.6.1 Wavelet Decomposition Fundamentals

The wavelet transform provides a time-frequency (or space-frequency) analysis of a signal by decomposing it into components localized in both the spatial and frequency domains. Unlike the Fourier transform, which projects a signal onto infinite sinusoids and loses all spatial localization, the wavelet transform uses basis functions of finite support that capture both *where* and *at what scale* features occur.

The **Continuous Wavelet Transform (CWT)** of a signal $f(t) \in L^2(\mathbb{R})$ is defined

with respect to a *mother wavelet* $\psi(t)$ [26]:

$$W_f(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} f(t) \psi^* \left(\frac{t - b}{a} \right) dt \quad (2-19)$$

where $a \in \mathbb{R} \setminus \{0\}$ is the scale (dilation) parameter, $b \in \mathbb{R}$ is the translation (shift) parameter, and ψ^* denotes the complex conjugate. The factor $|a|^{-1/2}$ ensures energy normalization across scales. Small values of a produce narrow, high-frequency wavelets that capture fine details, while large values of a produce wide, low-frequency wavelets that capture coarse structures. By varying both a and b , the CWT produces a two-dimensional representation of the signal in the scale-translation plane, revealing how frequency content evolves across spatial position.

The CWT suffers from redundancy: the transform of a one-dimensional signal yields a two-dimensional coefficient map, and neighboring scales and translations are highly correlated. For computational efficiency and practical signal processing, the wavelet transform is commonly discretized.

2.2.6.2 Discrete Wavelet Transform

The **Discrete Wavelet Transform (DWT)** replaces the continuous scale-translation parameters with discrete samples, most commonly on a dyadic grid where $a = 2^{-j}$ and $b = k \cdot 2^{-j}$ for integers $j, k \in \mathbb{Z}$ [27]. Under a multiresolution analysis framework [27], the DWT can be efficiently implemented as a cascade of filter banks. At each level, the signal is convolved with a pair of quadrature mirror filters: a low-pass filter $h[n]$ associated with the scaling function $\phi(t)$, and a high-pass filter $g[n]$ associated with the wavelet function $\psi(t)$, followed by downsampling by a factor of 2:

$$c_{j+1}[k] = \sum_n h[n - 2k] c_j[n], \quad d_{j+1}[k] = \sum_n g[n - 2k] c_j[n] \quad (2-20)$$

where c_j are the approximation coefficients at level j and d_j are the detail coefficients. The approximation c_{j+1} captures the low-frequency content at coarser resolution, while the detail d_{j+1} captures the high-frequency information lost during downsampling.

For a 2D signal such as an image, the DWT is applied separably: first filtering rows, then filtering columns of the intermediate result. This produces four subbands per decomposition level:

- **LL (Low-Low):** Approximation coefficients, obtained by applying the low-pass filter h to both rows and columns. This subband is a downsampled version of the input that

preserves the overall spatial layout.

- **LH** (Low-High): Horizontal detail coefficients, obtained by applying h to rows and g to columns, capturing vertical edges.
- **HL** (High-Low): Vertical detail coefficients, obtained by applying g to rows and h to columns, capturing horizontal edges.
- **HH** (High-High): Diagonal detail coefficients, obtained by applying g to both rows and columns, capturing diagonal edges and fine textures.

Each subband has half the spatial resolution of the input in both dimensions. The decomposition can be applied recursively to the LL subband to obtain multi-level wavelet representations, where each level captures features at a different spatial scale.

The key advantage of wavelet decomposition over standard downsampling (e.g., strided convolution or max pooling) is that it provides a principled frequency separation. Standard downsampling discards high-frequency information entirely, which can eliminate fine textures and edges critical for accurate boundary delineation. Wavelet decomposition preserves high-frequency details in the LH, HL, and HH subbands, making these details available for subsequent processing.

2.2.6.3 Wavelet-Based Downsampling in CNNs

Integrating DWT into convolutional neural networks replaces standard downsampling operations (strided convolution or max pooling) with wavelet-based downsampling. In this approach, a feature map at one resolution is decomposed into its four wavelet subbands using DWT. The LL subband serves as the downsampled feature map for the next network stage, while the detail subbands (LH, HL, HH) can be concatenated with or injected into intermediate layers to preserve high-frequency information.

The wavelet-based downsampling operation can be expressed as follows. Given a feature map $\mathbf{F} \in \mathbb{R}^{H \times W \times C}$, the DWT produces four subbands each of size $\frac{H}{2} \times \frac{W}{2} \times C$. The LL subband $\mathbf{F}_{LL}$ is passed to the next convolutional block as the primary feature representation. The detail subbands can be processed by lightweight convolutional layers and merged back into the feature pipeline through skip connections or attention mechanisms.

This design is particularly relevant for nuclei detection, where cell boundaries are defined by fine high-frequency edges in the H&E image. By preserving these details through the downsampling hierarchy, wavelet-based networks can maintain boundary-aware features at deeper layers where the detection head operates. WaveConvX [28] demonstrates this principle in the context of histopathology image classification, showing that wavelet integration improves feature quality for FCN-based architectures.

2.2.7 Star-Convex Object Representation

2.2.7.1 Geometric Definitions

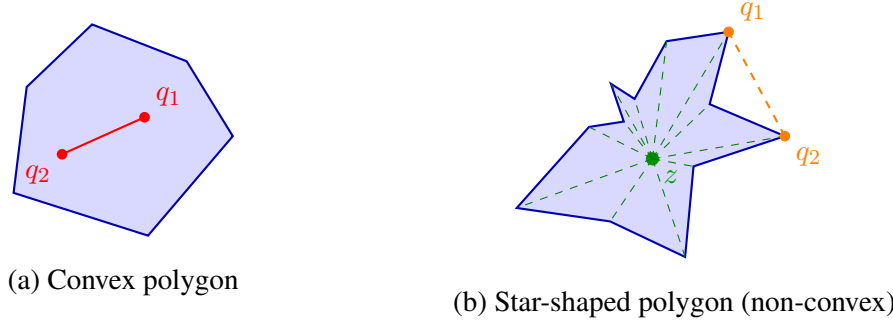


Figure 2.1. Illustration of convex and star-shaped polygons. (a) A convex polygon: the segment $\overline{q_1q_2}$ connecting any two interior points lies entirely within the polygon. (b) A star-shaped polygon that is not convex: the kernel point z (near the center) sees all boundary vertices (dashed green), yet the segment $\overline{q_1q_2}$ connecting two interior points exits the polygon, demonstrating that the interior is not a convex set.

A **convex polygon** is a polygon whose interior forms a convex set. A polygon P is convex if its interior region is a convex set [29]. A region R is a convex set if, for any two points q_1 and q_2 in R , the segment $\overline{q_1q_2}$ is entirely contained in R [29]. Intuitively, for a convex polygon, the line segment connecting any two points inside the polygon lies entirely within the polygon, and no interior angle exceeds 180° .

A **star-shaped polygon** generalizes the notion of convexity. A polygon P is star-shaped if there exists a point z not external to P such that for all points p of P , the line segment $\overline{zp}$ lies entirely within P [29]. The set of all such points z is called the *kernel* of the polygon. A star-shaped polygon can be viewed as one where there is at least one interior point visible to every other point in the polygon. Every convex polygon is star-shaped (the kernel is the entire interior), but a star-shaped polygon may have indentations that make it non-convex.

A **star-convex polygon** is a polygon P that is simultaneously star-shaped and convex. This dual property makes star-convex polygons particularly useful for representing cell nuclei: because they are convex, every boundary point can be uniquely described by a single ray emanating from the centroid, and because they are star-shaped, the centroid can be placed at a kernel point where all such rays intersect the boundary exactly once. In polar coordinates, a star-convex polygon centered at its centroid c can therefore be completely represented by a set of radial distances $\{d_0, d_1, \dots, d_{n-1}\}$ measured at n fixed angular intervals from the centroid to the boundary.

2.2.7.2 Suitability for Nuclei Representation

The star-convex polygon representation is well-suited for representing cell nuclei because of their characteristic morphology. The majority of cell nuclei encountered in H&E-stained tissue sections are approximately convex or roundish in shape [9]. Healthy epithelial nuclei are round or elliptical, fibroblasts and endothelial cells are elongated but convex, and even malignant nuclei with pleomorphic variations largely retain a convex silhouette in their two-dimensional projection. This near-ubiquitous convexity means that a star-convex polygon can capture the full boundary geometry of a nucleus with a compact set of radial distances, avoiding the information loss of axis-aligned bounding boxes while remaining far more parameter-efficient than dense per-pixel segmentation masks [8].

2.2.8 End-to-End Detection with Bipartite Matching

2.2.8.1 Bipartite Matching: Hungarian Algorithm

Bipartite matching establishes a one-to-one correspondence between two sets of elements by finding an optimal assignment that minimizes a total cost. In the context of object detection, bipartite matching assigns each ground-truth object to exactly one prediction and each prediction to at most one ground-truth object, eliminating the ambiguity inherent in IoU-based assignment where multiple predictions can match the same ground-truth object.

The assignment is formulated as a minimum-cost bipartite matching problem. Given a set of N ground-truth objects and M predictions, a cost matrix $\mathbf{C} \in \mathbb{R}^{N \times M}$ is constructed where $C_{i,j}$ measures the dissimilarity between ground-truth i and prediction j . A common choice is the negative IoU: $C_{i,j} = 1 - \text{IoU}(g_i, p_j)$. The optimal assignment is found by solving:

$$\begin{aligned}
& \min_{x_{i,j}} \quad \sum_{i=1}^N \sum_{j=1}^M C_{i,j} x_{i,j} \\
& \text{s.t.} \quad \sum_{j=1}^M x_{i,j} = 1, \quad \forall i \in \{1, \dots, N\} \\
& \quad \quad \sum_{i=1}^N x_{i,j} \leq 1, \quad \forall j \in \{1, \dots, M\} \\
& \quad \quad x_{i,j} \in \{0, 1\}
\end{aligned} \tag{2-21}$$

where $x_{i,j} = 1$ if ground-truth i is assigned to prediction j , and 0 otherwise. The first constraint ensures every ground-truth object receives exactly one match. The second constraint ensures no prediction is matched to more than one ground-truth object. When $M > N$, predictions not assigned to any ground-truth are treated as background.

The **Hungarian algorithm** (Kuhn–Munkres) solves this problem in $O(\max(N, M)^3)$ time through the following steps on a square cost matrix (padded with large values if $N \neq M$): (1) subtract the row minimum from each row, (2) subtract the column minimum from each column, (3) cover all zero entries with the minimum number of horizontal and vertical lines, (4) if the number of lines equals N , an optimal assignment is found by selecting independent zeros; otherwise, subtract the smallest uncovered value from all uncovered entries, add it to all doubly covered entries, and repeat from step 3.

This bipartite matching formulation is the foundation of end-to-end detection frameworks such as DETR and LSP-DETR [15]. By guaranteeing a unique assignment, it eliminates the need for NMS as a post-processing step, since each ground-truth object is matched to exactly one prediction and there are no duplicate assignments to suppress.

2.2.8.2 Task-Aligned Matching

Task-aligned matching is an assignment strategy used in YOLO-based detection frameworks that jointly considers classification and localization quality when matching predictions to ground-truth objects. Rather than using IoU alone, task-aligned matching defines a combined alignment metric:

$$t = s^\alpha \cdot u^\beta \quad (2-22)$$

where s is the classification confidence score, u is the predicted IoU between the prediction and the ground-truth object, and α, β are weighting hyperparameters that balance the contribution of classification and localization. A prediction is assigned to a ground-truth object only if its alignment metric t exceeds a threshold, and each ground-truth object is assigned to the prediction with the highest t value. This ensures that predictions assigned to an object are both well-localized (high IoU) and confidently classified, improving training stability.

2.2.8.3 Dual-Label Assignment

Dual-label assignment, introduced in YOLO26, is a training strategy that combines one-to-many and one-to-one matching within a single architecture to achieve the benefits of both paradigms. The model employs two parallel detection heads: a **one-to-many head** that assigns multiple predictions to each ground-truth object (as in conventional YOLO training), and a **one-to-one head** that enforces a unique bipartite matching between predictions and ground-truth objects (as in DETR-style training).

The two heads are balanced through an **annealing weight schedule** that controls their relative contribution to the total loss. During early training epochs, the one-to-many head dominates, providing dense gradient signals that stabilize training and accelerate

feature learning. As training progresses, the weight shifts gradually toward the one-to-one head, which refines the predictions to produce unambiguous, duplicate-free detections. This transition allows the model to benefit from the richer supervision of one-to-many matching early on while converging to the cleaner one-to-one assignment necessary for NMS-free inference. At inference time, only the one-to-one head is used, eliminating the need for non-maximum suppression.

2.2.9 Evaluation Metrics

2.2.9.1 Precision, Recall, and F1 Score

Object detection and segmentation are evaluated by comparing predicted instances against ground-truth annotations. Each prediction is classified as a **true positive (TP)** if it correctly matches a ground-truth object, a **false positive (FP)** if it does not correspond to any ground-truth object, or a **false negative (FN)** if a ground-truth object is missed. The definition of a correct match depends on the task: for bounding-box or polygon-based detection, a prediction is typically considered a TP if its intersection over union (IoU) with the ground truth exceeds a threshold τ (commonly $\tau = 0.5$); for centroid-based detection, a prediction may be considered a TP if the distance between the predicted and ground-truth centroids falls below a distance threshold.

Given the counts of TP, FP, and FN, precision and recall are defined as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2-23)$$

Precision measures the fraction of predictions that are correct (fewer FPs yield higher precision), while recall measures the fraction of ground-truth objects that are found (fewer FNs yield higher recall). The two are often in tension: a model that predicts conservatively achieves high precision but low recall, while a model that predicts aggressively achieves high recall but low precision.

The **F1 score** is the harmonic mean of precision and recall, providing a single balanced metric:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2-24)$$

2.2.9.2 Mean Average Precision (mAP)

Mean Average Precision (mAP) is the standard evaluation metric for object detection. It is computed by first calculating precision–recall curves for each object class and then averaging the area under these curves across all classes.

Given a set of detections ranked by confidence score and a chosen match criterion

(typically $\text{IoU} > 0.5$), precision and recall are computed at each confidence threshold to produce a precision–recall curve. The Average Precision (AP) for a single class is then the area under the interpolated precision–recall curve:

$$\text{AP} = \int_0^1 P_{\text{interp}}(R) dR \quad (2-25)$$

where $P_{\text{interp}}(R) = \max_{\tilde{R} \geq R} P(\tilde{R})$ is the interpolated precision at recall level R . In practice, this is approximated by evaluating precision at a discrete set of recall points.

The mean Average Precision is then the mean of per-class AP values:

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c \quad (2-26)$$

where C is the number of object classes. When reported at a specific IoU threshold (e.g., $\text{mAP}@0.5$), detections are matched using that threshold. The COCO evaluation protocol averages mAP over multiple IoU thresholds:

$$\text{mAP}@[.50:.05:.95] = \frac{1}{K} \sum_{k=1}^K \text{mAP}(\theta_k) \quad (2-27)$$

where $\theta_k \in \{0.50, 0.55, \dots, 0.95\}$ and $K = 10$. This provides a more comprehensive assessment of localization accuracy than evaluation at a single threshold. The notation $\text{mAP}@0.50$ (or AP_{50}) refers to evaluation at a single IoU threshold of 0.50.

2.2.9.3 Panoptic Quality (PQ)

Panoptic Quality (PQ) was introduced by Kirillov et al. [30] as a unified metric for panoptic segmentation, combining both recognition quality (detection accuracy) and segmentation quality (boundary accuracy). PQ has become the standard evaluation metric for nuclei instance segmentation benchmarks such as PanNuke [31] and CoNIC [32].

Given a ground truth segmentation G and a predicted segmentation P , each instance in G and P is uniquely labeled. A predicted segment $\hat{s}$ is matched to a ground truth segment s if and only if their intersection over union (IoU) exceeds a threshold, typically $\text{IoU} > 0.5$:

$$\text{IoU}(s, \hat{s}) = \frac{|s \cap \hat{s}|}{|s \cup \hat{s}|} \quad (2-28)$$

After matching, PQ is computed as:

$$\text{PQ} = \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{\text{Recognition Quality (RQ)}} \times \underbrace{\frac{\sum_{(s,\hat{s}) \in TP} \text{IoU}(s, \hat{s})}{|TP|}}_{\text{Segmentation Quality (SQ)}} \quad (2-29)$$

where TP is the set of true positive pairs (matched ground truth–prediction pairs), FP is the set of false positive predictions (unmatched predictions), and FN is the set of false negative ground truth segments (unmatched ground truth). PQ thus decomposes into two interpretable components:

- **Recognition Quality (RQ):** measures detection accuracy and is equivalent to the F1 score computed over matched and unmatched segments.
- **Segmentation Quality (SQ):** measures the average IoU of correctly matched pairs, quantifying how precisely the predicted boundaries align with the ground truth.

Several PQ variants are commonly reported in nuclei segmentation literature:

- **Instance PQ (iPQ):** evaluates PQ on a per-image basis and then averages across all images in the dataset. Each image contributes equally regardless of the number of nuclei it contains.
- **Multi-class PQ (mPQ):** extends PQ to multi-class settings by computing PQ independently for each nuclei type and then averaging across all classes:

$$\text{mPQ} = \frac{1}{C} \sum_{c=1}^C \text{PQ}_c \quad (2-30)$$

where C is the number of classes and PQ_c is the PQ computed for class c only.

- **Binary PQ (bPQ):** computes PQ in a binary setting where all nuclei types are treated as a single class, ignoring type-specific distinctions. This is useful for evaluating pure instance segmentation performance independent of classification accuracy.
- **PanNuke PQ (nuPQ):** the specific PQ variant used in the PanNuke benchmark, which computes mPQ across the 19 nuclei type categories defined in the dataset.
- **Masked PQ (MPQ):** a variant introduced by LSP-DETR [15] designed for fair evaluation of overlapping nuclei. Standard PQ penalizes polygon predictions that cross artificially separated ground-truth boundaries in regions of biological overlap. MPQ replaces the standard IoU with a **masked IoU (mIoU)** that isolates evaluation to the matched ground-truth instance. Given a ground-truth mask m_k and predicted polygon $\hat{m}_k$, and letting $\mathcal{G} = \bigcup_{k=1}^K m_k$ be the union of all ground-truth masks:

$$\text{mIoU}(m_k, \hat{m}_k) = \frac{|m_k \cap \hat{m}_k|}{|m_k \cup (\hat{m}_k \setminus \mathcal{G})|} \quad (2-31)$$

$$\text{MPQ} = \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{\text{RQ}} \times \underbrace{\frac{\sum_{(m_k, \hat{m}_k) \in TP} \text{mIoU}(m_k, \hat{m}_k)}{|TP|}}_{\text{SQ}_{\text{mask}}} \quad (2-32)$$

The denominator in mIoU includes the ground-truth area plus only those parts of the prediction that fall outside $\mathcal{G}$, meaning predictions overlapping *other* ground-truth masks beyond the matched one are penalized, but overlap with the matched instance itself is not double-counted. This makes MPQ the appropriate metric for polygon-based methods that output realistically overlapping cell representations.

2.2.9.4 Aggregated Jaccard Index (AJI)

The Aggregated Jaccard Index (AJI) [33] is an instance-level segmentation metric designed to address the overcounting problem in the standard Jaccard Index when applied to multi-object segmentation. In nuclei segmentation, the standard Jaccard Index (Intersection over Union, IoU) computed per instance and then averaged can overestimate performance because each ground-truth nucleus is free to match with any predicted nucleus independently, and unmatched predictions do not penalize the score.

AJI resolves this by computing a single global intersection-over-union across all instances. Given a set of ground-truth nuclei $G = \{g_1, g_2, \dots, g_N\}$ and predicted nuclei $P = \{p_1, p_2, \dots, p_M\}$, AJI first finds the optimal one-to-one assignment that maximizes total pairwise IoU. Let $\mathcal{M} \subseteq G \times P$ denote the set of matched pairs, $G_{\text{matched}} \subseteq G$ the set of matched ground-truth nuclei, and $P_{\text{matched}} \subseteq P$ the set of matched predicted nuclei. The AJI is then defined as:

$$\text{AJI} = \frac{\sum_{(g_i, p_j) \in \mathcal{M}} |g_i \cap p_j|}{\sum_{(g_i, p_j) \in \mathcal{M}} |g_i \cup p_j| + \sum_{g_k \in G \setminus G_{\text{matched}}} |g_k| + \sum_{p_l \in P \setminus P_{\text{matched}}} |p_l|} \quad (2-33)$$

The numerator sums the intersection area over all matched pairs, while the denominator sums the union area over all matched pairs plus the areas of all unmatched ground-truth and predicted nuclei. This formulation penalizes both false negatives (unmatched ground-truth nuclei) and false positives (unmatched predictions), providing a more stringent evaluation than per-instance averaging.

CHAPTER III

RESEARCH METHODOLOGY

3.1 Datasets and Environmental Setup

This study used publicly available secondary datasets for training and evaluation. Table 4.1 summarized the datasets and their roles.

Table 3.1. Summary of datasets used in this study.

Dataset	Purpose	Tissue	Format
PanNuke	Primary benchmarking	Multi-organ	Pickled bytes
PUMA	TILs detection	Melanoma	TIFF, GeoJSON
panopTILs	TILs detection	Breast	PNG
MoNuSAC	TILs detection	Multi-organ	Pickled bytes

3.1.1 Datasets

3.1.1.1 PanNuke

PanNuke [31, 34] is a large-scale pan-cancer nucleus instance segmentation dataset containing approximately 189,744 labeled nuclei across 19 tissue types. Each nucleus is annotated with an instance mask and one of five nuclear categories. The dataset is provided as pickled bytes containing image patches and corresponding label masks. In this study, PanNuke served as the primary benchmarking dataset for evaluating nuclei detection and segmentation performance.

3.1.1.2 PUMA

PUMA (Melanoma Histopathology Dataset with Tissue and Nuclei Annotations) [35] provides annotated histopathology images for TILs detection in melanoma tissue. The dataset is distributed as TIFF images paired with GeoJSON annotation files containing polygon-level cell boundary annotations. This dataset was used to evaluate RayCastED's ability to detect and delineate TILs in a clinically relevant setting.

3.1.1.3 panopTILs

panopTILs [36] provides TILs annotations on breast cancer histopathology images. The dataset is distributed as PNG image patches with corresponding annotation files. This dataset supplemented the TILs evaluation alongside PUMA, providing breast tissue samples for assessing TILs detection performance across cancer types.

3.1.1.4 MoNuSAC

MoNuSAC [37] provides annotated nuclei across multiple organ types with four nuclear categories: epithelial, lymphocyte, neutrophil, and macrophage. The dataset is provided as pickled bytes. The lymphocyte annotations in MoNuSAC were particularly relevant for evaluating TILs detection, as lymphocytes are the primary cell type of interest in TILs scoring.

3.1.2 Environmental Setup

3.2 Methods

3.2.1 Data Preprocessing

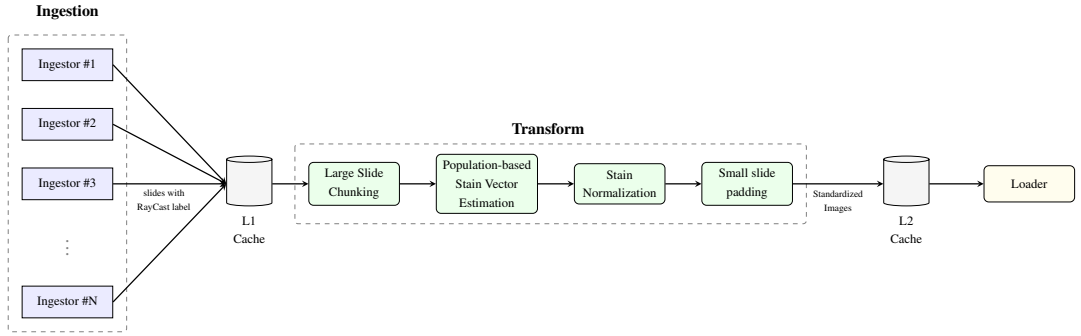


Figure 3.2. High-level Pipeline Design

3.2.1.1 Ingestion Pipeline

3.2.1.1.1 RayCast Representation.

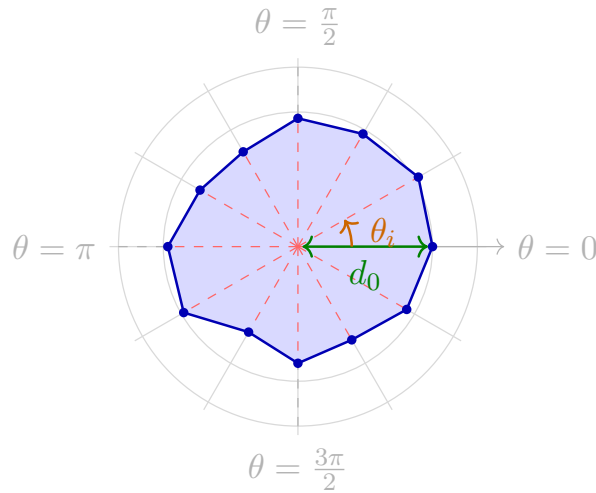


Figure 3.3. A star-convex polygon with centroid c , polar grid, and 12 radial rays with angular spacing θ_i .

Encoding.

Given a closed polygon $\mathcal{P}$ defined by an ordered set of K vertices $\{\mathbf{V}_j\}_{j=0}^{K-1}$ in $\mathbb{R}^2$ and a class label c , the RayCast representation encodes the polygon as a fixed-length vector by expressing its boundary as radial distances from a chosen centroid along R uniformly spaced angular directions. The output is an annotation vector:

$$\mathbf{a} = (c, c_x, c_y, d_0, d_1, \dots, d_{R-1})^\top \in \mathbb{R}^{3+R} \quad (3-1)$$

where (c_x, c_y) is the centroid, R is the number of rays (set to 32 in this study, and constrained to be a multiple of 4), and d_r records the Euclidean distance from the centroid to the polygon boundary along ray r at angle $\theta_r = r \cdot 2\pi/R$. The convention is $\theta_0 = 0$ (east, $+X$ axis), increasing counter-clockwise. Figure 3.4 illustrates the concept.

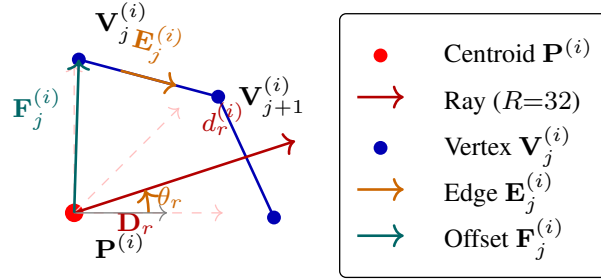


Figure 3.4. RayCast representation of a cell polygon. From the area-weighted centroid $\mathbf{P}$, $R = 32$ rays are cast at uniformly spaced angles. Each distance d_r records the nearest boundary intersection along ray r .

The generation algorithm proceeds in four stages: (1) centroid computation, (2) containment verification and exterior centroid correction, (3) ray–edge intersection solving, and (4) nearest-hit selection and output assembly. The entire pipeline is formulated as batched tensor operations for GPU parallelism, processing all polygons in a region of interest simultaneously.

Raster Mask to Contour Extraction. Datasets such as PanNuke and MoNuSAC provide per-cell binary raster masks rather than polygon coordinates. For each cell, the binary mask is decoded and contour vertices are extracted via Moore-neighbor tracing (OpenCV’s `findContours` with `CHAIN_APPROX_NONE`) to preserve full boundary fidelity. When a mask produces multiple contours, only the contour with the largest enclosed area is retained. Contours with fewer than 3 vertices are discarded.

Centroid Computation. The area-weighted centroid serves as the origin from which all rays emanate. Given the polygon vertices with cyclic indexing $\mathbf{V}_K \equiv \mathbf{V}_0$, the signed area is computed via the shoelace formula:

$$A = \frac{1}{2} \sum_{j=0}^{K-1} (x_j y_{j+1} - x_{j+1} y_j) \quad (3-2)$$

Polygons with $|A| < \varepsilon$ (where $\varepsilon = 10^{-12}$) are near-zero area; in this case the centroid is set to the first vertex $\mathbf{V}_0$. The area-weighted centroid coordinates are:

$$c_x = \frac{1}{6A} \sum_{j=0}^{K-1} (x_j + x_{j+1}) (x_j y_{j+1} - x_{j+1} y_j) \quad (3-3)$$

$$c_y = \frac{1}{6A} \sum_{j=0}^{K-1} (y_j + y_{j+1}) (x_j y_{j+1} - x_{j+1} y_j) \quad (3-4)$$

For GPU-batched processing across N polygons padded to $K_{\max} = \max_i K_i$ vertices, an edge mask $\mathbf{M}^{\text{edge}} \in \{0, 1\}^{N \times K_{\max}}$ distinguishes real polygon edges from padding artifacts (the closing edge $j = K_i - 1$ is included). The cross-product, area, and centroid are computed element-wise and reduced over the vertex dimension in a single kernel launch.

Containment Verification and Exterior Centroid Correction. For convex polygons the area-weighted centroid always lies inside the boundary. For the mildly concave nuclei encountered in histopathology, it may fall outside. A crossing-number point-in-polygon test is applied: a horizontal ray is cast in the $+X$ direction, and the number of edge crossings determines interiority (odd crossings = inside).

When the centroid falls outside the polygon, a binary search positions it at an interior point. The target $\mathbf{T}$ is the mean of the three real vertices closest to the exterior centroid in Euclidean distance:

$$\mathbf{T} = \frac{1}{3} (\mathbf{V}_{j_1} + \mathbf{V}_{j_2} + \mathbf{V}_{j_3}) \quad (3-5)$$

The algorithm iteratively probes the line segment $[\mathbf{P}, \mathbf{T}]$ at fractions $\alpha \in \{0.5, 0.75, 0.875, 0.9375\}$ testing each candidate $\mathbf{Q}_\alpha = \mathbf{P} + \alpha(\mathbf{T} - \mathbf{P})$ for interiority via the crossing-number test. The candidate is updated only if the test succeeds. After 5 iterations the final precision is $1 - 0.96875 \approx 3.1\%$ of $\|\mathbf{T} - \mathbf{P}\|$. In practice, histopathology nuclei exhibit at most mild concavity, so the area-weighted centroid is interior in nearly all cases and the fallback is rarely triggered.

Ray-Edge Intersection. The core computation solves for the intersection of each ray with every polygon edge. The ray from centroid $\mathbf{P}$ in unit direction $\mathbf{D}_r = (\cos \theta_r, \sin \theta_r)^\top$ is parameterized as:

$$\mathbf{L}_r(t) = \mathbf{P} + t \mathbf{D}_r, \quad t \geq 0 \quad (3-6)$$

An edge segment from $\mathbf{V}_j$ to $\mathbf{V}_{j+1}$ with edge vector $\mathbf{E}_j = \mathbf{V}_{j+1} - \mathbf{V}_j$ is:

$$\mathbf{Q}_j(s) = \mathbf{V}_j + s \mathbf{E}_j, \quad s \in [0, 1] \quad (3-7)$$

Setting $\mathbf{L}_r(t) = \mathbf{Q}_j(s)$ and defining the offset $\mathbf{F}_j = \mathbf{V}_j - \mathbf{P}$ yields a 2×2 linear system:

$$\underbrace{\begin{bmatrix} \cos \theta_r & -E_{j,x} \\ \sin \theta_r & -E_{j,y} \end{bmatrix}}_{\mathbf{M}_{rj}} \begin{bmatrix} t \\ s \end{bmatrix} = \underbrace{\begin{bmatrix} F_{j,x} \\ F_{j,y} \end{bmatrix}}_{\mathbf{b}_j} \quad (3-8)$$

The system is solved via Cramer's rule using the 2D cross product $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x$:

$$\det_{rj} = \det(\mathbf{M}_{rj}) = -\cos \theta_r \cdot E_{j,y} + \sin \theta_r \cdot E_{j,x} \quad (3-9)$$

$$t_{rj} = \frac{-\mathbf{F}_j \times \mathbf{E}_j}{\det_{rj}}, \quad s_{rj} = \frac{\mathbf{D}_r \times \mathbf{F}_j}{\det_{rj}} \quad (3-10)$$

The right-hand side $\mathbf{b}_j$ is independent of the ray direction; only the first column of $\mathbf{M}_{rj}$ varies with r . Each (r, j) pair is solved independently, enabling full data-parallelism over all three dimensions (polygon, ray, edge) simultaneously.

Validity Filtering and Nearest-Hit Selection. An intersection is valid if and only if: (1) $|\det_{rj}| > \epsilon_d$ (ray is not parallel to the edge, $\epsilon_d = 10^{-10}$), (2) $t_{rj} > 0$ (intersection lies in front of the centroid; the strict inequality prevents self-intersection artifacts when the centroid lies exactly on the boundary), (3) $-\epsilon \leq s_{rj} \leq 1 + \epsilon$ (intersection falls on the edge segment, $\epsilon = 10^{-6}$), and (4) the edge is a real polygon edge (not a padding artifact). Degenerate cases are handled as follows:

- **Ray through shared vertex:** both adjacent edges produce valid hits with the same t ; the minimum selection naturally resolves this.
- **Ray tangent to edge:** filtered by the determinant threshold $|\det_{rj}| < \epsilon_d$.
- **Ray parallel but offset:** produces $t \rightarrow \pm\infty$, filtered by the $t > 0$ condition.
- **Ray collinear with edge:** the edge lies along the ray direction; d_r is conservatively set to $d_r = 0$.

For each ray r , the boundary distance is the minimum positive t among valid

intersections:

$$d_r = \min_{j \in \mathcal{V}_r} t_{rj}, \quad \mathcal{V}_r = \{j : \text{valid}_{rj} = \text{True}\} \quad (3-11)$$

Since $\mathbf{D}_r$ is a unit vector, $d_r = t_{rj^*}$ is directly the Euclidean distance from the centroid to the boundary intersection. Rays with no valid intersection ($\mathcal{V}_r = \emptyset$) are assigned $d_r = 0$, following the convention that zero distance indicates a missed ray.

Batched GPU Implementation. The entire pipeline is implemented as vectorized tensor operations in PyTorch, processing all N polygons in a region of interest simultaneously on GPU. Polygons are padded to $K_{\max}$ vertices by repeating the last vertex; past the real vertex count, the padding creates degenerate zero-length edges that naturally fail the determinant check and are excluded from results.

The core intersection solve operates over the full $(N, R, K_{\max})$ tensor. Input shapes are broadcast as follows: vertex coordinates $(N, 1, K_{\max})$, centroid $(N, 1, 1)$, ray trig functions $(1, R, 1)$, expanding element-wise to $(N, R, K_{\max})$. All Cramer’s rule computations, validity masking, and the final min-reduction over the edge dimension execute in a single kernel launch without explicit loops.

The computational work scales as $O(N \cdot R \cdot K_{\max})$. The only serial component is the exterior centroid fallback (5 binary search iterations), which operates on a negligible fraction of the batch since histopathology nuclei rarely produce exterior centroids. For a typical region of interest ($N \approx 1000$, $R = 32$, $K_{\max} \approx 500$), the intersection tensors require approximately $4 \cdot N \cdot R \cdot K_{\max} \approx 64\text{MB}$ each, well within modern GPU memory.

Decoding. The raycast representation decodes back to a polygon by reconstructing vertex positions from the centroid and radial distances:

$$\mathbf{V}_r = \mathbf{P} + d_r \cdot \mathbf{D}_r, \quad r = 0, \dots, R-1 \quad (3-12)$$

connected cyclically. In batched form for N annotations with distance matrix $\mathbf{D} \in \mathbb{R}^{N \times R}$ and precomputed direction vectors $\mathbf{c} = (\cos \theta_0, \dots, \cos \theta_{R-1})$ and $\mathbf{s} = (\sin \theta_0, \dots, \sin \theta_{R-1})$:

$$\mathbf{V}_x = c_x[:, \text{None}] + \mathbf{D} \odot \mathbf{c}^\top, \quad \mathbf{V}_y = c_y[:, \text{None}] + \mathbf{D} \odot \mathbf{s}^\top \quad (3-13)$$

yielding vertex coordinates $\mathbf{V} \in \mathbb{R}^{N \times R \times 2}$. The angular constants are precomputed once and shared across ingestion, model inference, loss computation, and evaluation.

3.2.1.1.2 Data Ingestion Design.

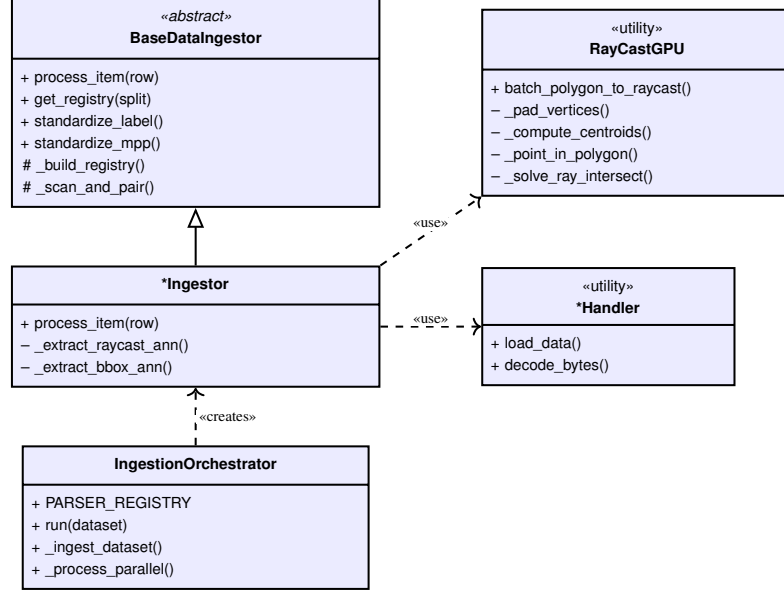


Figure 3.5. UML class diagram of the modular ingestion pipeline. `BaseDataIngestor` defines the shared infrastructure; concrete ingestors implement format-specific `process_item` logic; all converge on the unified raycast annotation format via `RayCastGPU`.

A practical computational pathology system must ingest annotations from heterogeneous source formats: binary instance masks, polygon coordinates, comma-separated coordinate strings, and integer label maps. Rather than implementing a monolithic converter per dataset, the ingestion pipeline follows a modular architecture with three design principles.

First, **format-agnostic core logic** (centroid computation, ray–edge intersection solving, MPP harmonization, and label translation) is shared across all ingestors. Second, each concrete ingestor implements only the **format-specific extraction** logic that parses its source format into the common intermediate representation: polygon vertex arrays with class labels. Third, **configuration-driven routing** specifies the choice of ingestor, split strategy, modality pairing, and label mapping declaratively, without code changes.

The architecture is built around an abstract base class that defines the contract every ingestor must fulfill: for each region of interest, produce a unified output $(id_{\text{roi}}, \mathbf{I}, \mathbf{A}, t_{\text{tissue}})$ consisting of an identifier, an RGB image, a raycast annotation matrix $\mathbf{A} \in \mathbb{R}^{N \times (3+R)}$, and a tissue provenance label. The sole abstract method is `process_item`, which each concrete ingestor implements for its specific format.

The base class provides a template method for **registry construction** in three phases: (1) file discovery and pairing, where image files are matched with annotation files using either bundled-archive (same file) or physical-parallel (paired directories)

strategies; (2) split assignment, where each pair is assigned to train/val/test using physical subdirectory names, filename regex patterns, or stratified random sampling by tissue type; and (3) registry assembly into a work queue. A **two-step label translation** decouples source vocabularies from the global integer encoding: a namespace map $\mathcal{N}$ translates raw labels to standardized strings shared across all datasets, and a global cell map $\mathcal{G}$ maps standardized strings to integer class IDs. Adding a new dataset requires only a new namespace map. Similarly, all annotations are rescaled to a common target microns-per-pixel via **MPP harmonization**, scaling image dimensions and annotation coordinates uniformly by the ratio $\rho = \sigma_{\text{native}} / \sigma_{\text{target}}$.

Concrete ingestors differ only in how they extract polygon vertices: mask-based sources decode compressed byte strings via Moore-neighbor contour tracing, while vector-based sources read polygon coordinates directly from geometry arrays or comma-separated coordinate strings requiring no rasterization. Mask-based ingestors process ROIs in parallel using a process pool, deferring GPU ray-cast conversion to the main process to avoid CUDA cross-process deadlock. All ingestors converge through a single conversion entry point, `RayCastGPU.batch_polygon_to_raycast`, which receives the format-independent intermediate representation (variable-length vertex arrays and class labels) and produces the unified raycast annotation **A**.

The `IngestionOrchestrator` governs the end-to-end process: a string-keyed registry maps configuration keys to concrete ingestor classes, validated at both config-load time and runtime. A two-layer YAML hierarchy (global defaults overridden per dataset) enables declarative specification of all parameters. Row-level errors are caught and logged without terminating the remaining dataset, so a single corrupt ROI does not halt ingestion. This separation of concerns means that adding a new dataset typically requires only a YAML configuration entry with a namespace map and format key, while the computational core remains invariant across all sources.

3.2.1.2 Transform Pipeline

3.2.1.2.1 Data Profile Registration.

Before any per-tile transformation is applied, the ingestion pipeline registers a data profile for each image in the dataset. This profile captures three categories of metadata that downstream transforms consume: (1) **tissue type and dataset source**, recorded as a string identifier and integer encoding, enabling stratified sampling across datasets and tissue categories during population-level stain estimation; (2) **native image dimensions** (W, H) and **microns-per-pixel** (MPP), required by the size standardization step to compute chopping and padding parameters; and (3) **tile-level stain statistics**, including a boolean flag indicating whether the tile contains sufficient tissue content (at least 100 non-white pixels) to be eligible for stain vector estimation. Profiles are assembled

during the ingestion registry phase and serialized alongside the raycast annotations, so that the transform pipeline can load them without re-reading the source images. This separation ensures that the transform pipeline operates purely on pre-computed metadata rather than performing redundant I/O.

3.2.1.2.2 Stain Vector Estimation and Normalization.

Histopathology images exhibit substantial color variation across scanners, staining batches, and laboratories. A model trained on darkly-stained slides may perform poorly on lightly-stained ones. The transform pipeline addresses this through a three-phase process. First, images are chopped into $S \times S$ tiles. Second, a subset of tiles is Pareto-sampled, the Macenko method [18] estimates per-tile stain vectors, and the results are aggregated into a population-level profile. Third, every tile is normalized against this profile and padded to a uniform size.

Population-Level Stain Vector Estimation.

Rather than processing every tile – which wastes computation and allows large datasets to dominate – stratified Pareto sampling is applied following Agraz et al. [38], who showed that 20% of a cohort suffices for a representative profile. The chunked registry is partitioned into strata by dataset and tissue type, ensuring each dataset contributes proportionally and rare tissue types are represented. Within each stratum containing n_j tiles, the number sampled is:

$$k_j = \max(1, \lceil 0.20 \cdot n_j \rceil) \quad (3-14)$$

Tiles are drawn uniformly at random with a fixed seed. The estimation set $\mathcal{S}$ is the union across all J strata.

For each sampled tile $k \in \mathcal{S}$, the Macenko method estimates a per-tile stain matrix $\mathbf{M}_{\text{stain}}^{(k)} \in \mathbb{R}^{2 \times 3}$ and maximum concentrations $\mathbf{c}_{\text{max}}^{(k)} \in \mathbb{R}^2$. The population profile is aggregated across the K tiles in $\mathcal{S}$ that yield valid estimates. The stain matrix is averaged since stain directions vary smoothly, while maximum concentrations use the 99th percentile to establish a robust upper bound:

$$\overline{\mathbf{M}} = \frac{1}{K} \sum_{k \in \mathcal{S}} \mathbf{M}_{\text{stain}}^{(k)}, \quad \mathbf{c}_{\text{max}}^* = P_{99}(\{\mathbf{c}_{\text{max}}^{(k)}\}_{k \in \mathcal{S}}) \quad (3-15)$$

Tiles that fail estimation (all-white or fewer than 100 tissue pixels) are silently excluded from K . The profile is serialized as JSON containing $\overline{\mathbf{M}}$, $\mathbf{c}_{\text{max}}^*$, the estimation method, and the tile count K for reuse during normalization and at inference time.

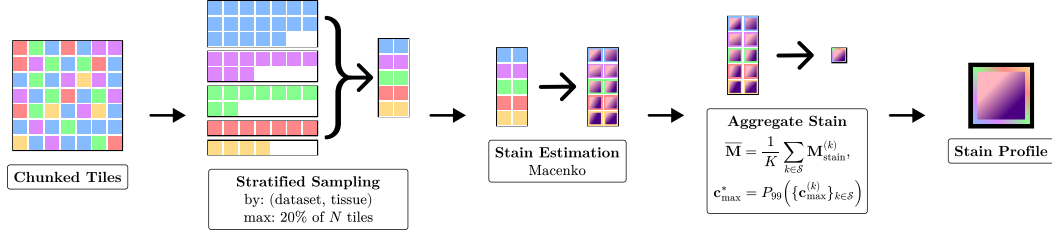


Figure 3.6. Population-level stain vector estimation. A stratified 20% subset of tiles is sampled per dataset and tissue type; the Macenko method estimates per-tile stain matrices and maximum concentrations, which are aggregated into the population profile $(\bar{M}, c_{\text{max}}^*)$.

Per-Tile Stain Normalization.

Given the population target $(\bar{M}, c_{\text{max}}^*)$ and a per-tile source estimate $(M_{\text{source}}, c_{\text{source}})$, normalization consists of five steps.

The tile is first converted from RGB to optical density space via the Beer–Lambert law:

$$\text{OD} = -\log_{10}\left(\frac{I_0 + 1}{I_o}\right) \quad (3-16)$$

where the $+1$ avoids $\log(0)$ at black pixels. The OD image is then decomposed into Hematoxylin and Eosin stain concentration maps using the pseudoinverse of the source stain matrix:

$$C_{\text{source}} = \text{OD} \cdot M_{\text{source}}^+ \quad (3-17)$$

Each channel’s concentration is rescaled to match the population target range by the per-channel percentile ratio:

$$C_{\text{norm}} = C_{\text{source}} \odot \frac{c_{\text{max}}^*}{c_{\text{source}}} \quad (3-18)$$

A small epsilon (10^{-6}) prevents division by zero when a concentration channel is empty. The normalized OD is reconstructed using the population mean stain directions, replacing the source-specific color signature with the standardized one:

$$\text{OD}_{\text{norm}} = C_{\text{norm}} \cdot \bar{M} \quad (3-19)$$

Finally, the normalized OD is converted back to RGB and clipped to $[0, 255]$:

$$I_{\text{norm}} = I_o \cdot 10^{-\text{OD}_{\text{norm}}} - 1 \quad (3-20)$$

The full transform can be expressed compactly as:

$$I_{\text{norm}} = I_o \cdot 10^{-\left(\text{OD} \cdot \mathbf{M}_{\text{source}}^+ \odot \frac{\mathbf{c}_{\text{max}}^*}{\mathbf{c}_{\text{source}}}\right) \cdot \overline{\mathbf{M}}} - 1 \quad (3-21)$$

3.2.1.2.3 Image Size Standardization.

Every ingested image must be reduced to a fixed $S \times S$ canvas (defaulting to 256×256 pixels) suitable for batched GPU training. Two operations handle the two possible regimes: images larger than S in either dimension are **chopped** into overlapping tiles; images smaller than or equal to S are **padded** with white pixels at the bottom and right edges.

Chopping: Dynamic Overlap Distribution.

For an image dimension $L > S$ with minimum overlap fraction α_{overlap} , the number of tiles n is computed from the effective stride $\sigma = S - \lfloor S \cdot \alpha_{\text{overlap}} \rfloor$. Tile starting positions are distributed uniformly via $\text{linspace}(0, L - S, n)$, which distributes any excess overlap evenly across all seams rather than concentrating it at the boundary.

Annotation-Aware Raycast Splitting.

Each cell is assigned to exactly one tile via a centroid-owns-cell policy: a cell belongs to the tile containing its centroid (c_x, c_y) . The splitting procedure has four steps. First, centroids are filtered to retain only cells within the tile bounds. Second, centroid coordinates are shifted to crop-relative space: $c'_x = c_x - x_0$, $c'_y = c_y - y_0$. Third, for each of the R rays, the distance to each of the four tile boundaries is computed. For a ray in direction $\mathbf{D}_r = (\cos \theta_r, \sin \theta_r)$, the boundary distances are:

$$d_{\text{right}} = \frac{w - c'_x}{\cos \theta_r}, \quad d_{\text{left}} = \frac{-c'_x}{\cos \theta_r}, \quad d_{\text{bottom}} = \frac{h - c'_y}{\sin \theta_r}, \quad d_{\text{top}} = \frac{-c'_y}{\sin \theta_r} \quad (3-22)$$

where terms with $|\cos \theta_r| \leq \varepsilon$ or $|\sin \theta_r| \leq \varepsilon$ are set to $+\infty$ (the ray is parallel to that boundary). The maximum permissible distance is $d_{\text{max}} = \min(d_{\text{right}}, d_{\text{left}}, d_{\text{bottom}}, d_{\text{top}})$, and each ray is clipped: $d'_r = \min(d_r, d_{\text{max}})$. Fourth, cells that lose too many rays to boundary clipping are discarded via a survival rate filter:

$$\frac{|\{r : d'_r > 0\}|}{R} \geq \tau_{\text{survival}} \quad (3-23)$$

where $\tau_{\text{survival}} = 0.5$. This prevents heavily truncated cells with unreliable shape representations from entering the training data. Figure 3.7 illustrates the centroid-owns-cell policy and ray clipping procedure.

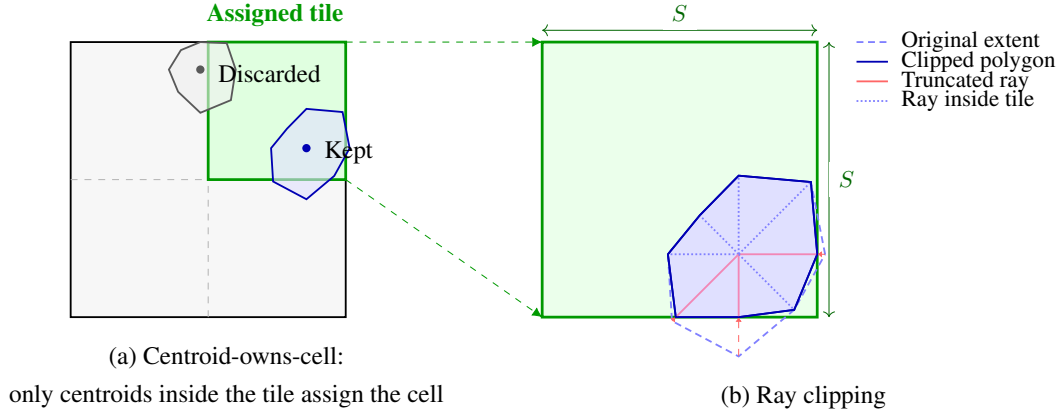


Figure 3.7. Annotation-aware raycast splitting. (a) The ROI is divided into overlapping tiles. Only cells whose centroids fall inside a given tile are assigned to it (centroid-owns-cell policy). (b) For each assigned cell, rays extending beyond the tile boundary are clipped to the nearest wall.

Padding.

After stain normalization, tiles with content smaller than $S \times S$ are padded with white pixels at the bottom and right edges. White padding is chosen because H&E slide backgrounds are white, so the model sees the same signal at tile edges as at slide borders. Bottom-right placement preserves annotation validity: the top-left origin $(0, 0)$ remains unchanged, so centroid positions and ray distances require no modification. The original content dimensions are recorded so that the training data loader can constrain random crop origins to the tissue-containing region.

3.2.1.3 Data Augmentation Strategies

3.2.1.3.1 RayCast Permutations.

Standard detection augmentations assume axis-aligned bounding boxes. Raycast annotations require specialized handling: spatial transforms (flips, rotations) permute the ray distance vector without changing magnitudes; scale multiplies all distances uniformly; and crops require ray clipping and a survival-rate filter to discard cells whose shape becomes unreliable.

All ray permutations depend only on R and the transform type, so they are pre-computed at configuration time:

Table 3.2. Precomputed ray permutation indices for each geometric transform.

Transform	Permutation	Example ($R = 32$)
Horizontal flip	$\pi_{\text{flip-h}}(r) = (R/2 - r) \bmod R$	ray 0 $\rightarrow$ 16
Vertical flip	$\pi_{\text{flip-v}}(r) = (R - r) \bmod R$	ray 0 $\rightarrow$ 0, ray 1 $\rightarrow$ 31
Rotate 90° CCW	$\pi_{\text{rot}}(r; 1) = (r + R/4) \bmod R$	ray 0 $\rightarrow$ 8
Rotate 180°	$\pi_{\text{rot}}(r; 2) = (r + R/2) \bmod R$	ray 0 $\rightarrow$ 16
Rotate 270° CCW	$\pi_{\text{rot}}(r; 3) = (r + 3R/4) \bmod R$	ray 0 $\rightarrow$ 24

Any augmentation that changes the spatial relationship between annotations and the canvas triggers a four-step clipping pipeline: (1) discard cells whose centroid falls outside the crop region, (2) translate surviving centroids to crop-relative coordinates, (3) for each ray, compute the distance to the four canvas boundaries and clip the ray to $d'_r = \min(d_r, d_{\max}(r))$, and (4) discard cells with fewer than $\tau_{\text{surv}} = 0.3$ surviving rays. The boundary distance $d_{\max}(r)$ is computed identically to the chopping procedure described above, using the same ray-by-ray intersection with tile edges.

3.2.1.3.2 Augmentation Operations.

The following augmentations are applied during training:

Random crop. Crop origin is constrained to the tissue-containing region to avoid sampling white padding. Annotations whose centroid falls outside the crop are dropped; remaining cells are translated to crop-relative space and rays are clipped.

Stain jitter. A photometric-only augmentation in HSV space that mimics inter-slide stain variation without modifying any annotation field:

$$H' = H + \delta_H, \quad S' = S \cdot \gamma_S, \quad V' = V \cdot \gamma_V \quad (3-24)$$

where $\delta_H \sim \mathcal{U}[-9^\circ, 9^\circ]$, $\gamma_S \sim \mathcal{U}[0.7, 1.3]$, and $\gamma_V \sim \mathcal{U}[0.8, 1.2]$. An optional Gaussian blur is applied with probability 0.2 ($\sigma \sim \mathcal{U}[0.5, 1.0]$).

Random scale. Isotropic scaling by $s \sim \mathcal{U}[0.7, 1.3]$: all spatial quantities (centroid, all ray distances) are multiplied by s . For $s > 1$, a random crop restores the image to $S \times S$; for $s \leq 1$, random padding places the scaled image on a white canvas. Boundary clipping follows.

Random translation. Shift by $(\Delta x, \Delta y)$ with $|\Delta x|, |\Delta y| \leq \lfloor 0.1 \cdot S \rfloor$. Only centroids move (ray distances are translation-invariant), followed by boundary clipping.

Horizontal flip. Applied with probability 0.5: centroid $c'_x = S - c_x$, and ray

distances are permuted by the precomputed flip index.

3.2.2 Architecture Design

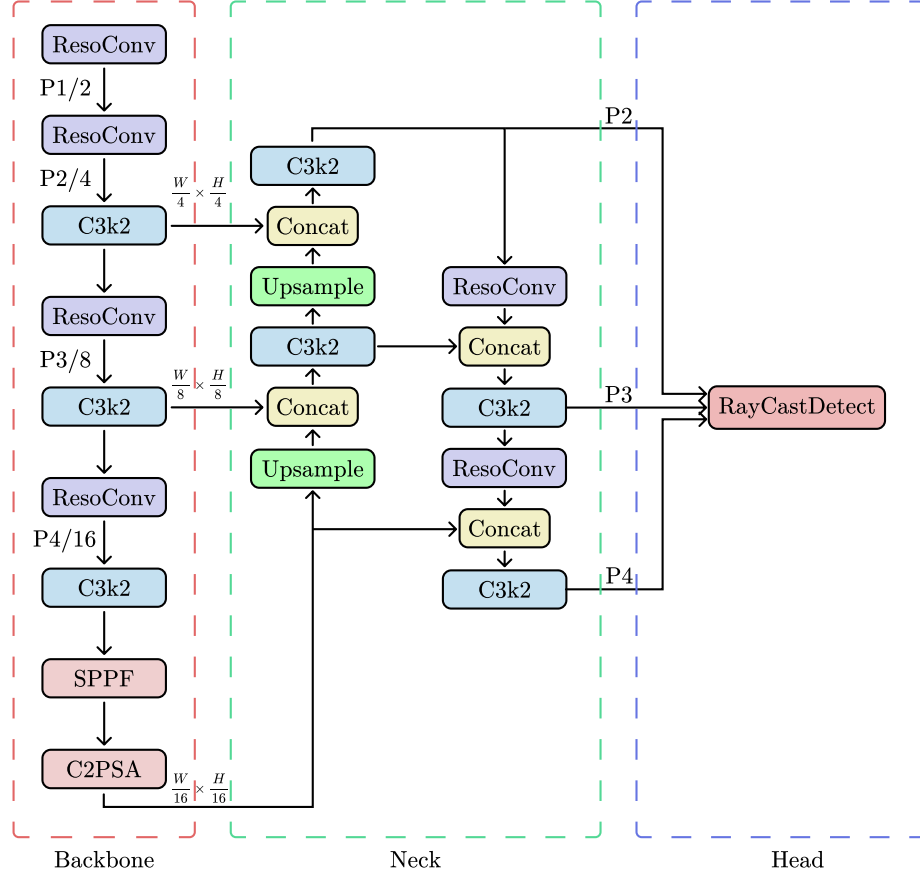


Figure 3.8. RayCastED model architecture

3.2.2.1 Design Considerations

Contour-based instance segmentation shares its architectural structure with object detection rather than with mask-based segmentation. Both contour-based instance segmentation and detection operate on an anchor grid, predict a sparse set of geometric parameters per location, and train through matching mechanisms rather than per-pixel classification. The star-convex ray representation used in this work replaces the bounding box regression target with polygon boundary distances. This is a change to the regression target, not to the underlying prediction paradigm, making the mature detection architecture ecosystem directly applicable. Four requirements constrain the base model choice: a fully convolutional backbone for lightweight inference, single-stage prediction for computational efficiency, an established architecture for reproducibility, and NMS-free operation to avoid the density-dependent counting bias described in Section ?? [TODO]. YOLO26 satisfies

all four through its dual-label assignment strategy ?? [TODO], which enables one-to-one matching at inference without post-processing. Its implementation within the Ultralytics framework further provides optimized training pipelines and export pathways for edge deployment.

YOLO26 is designed for general objects and employs feature pyramid levels P3 through P5 (strides 8, 16, and 32). For a 256×256 input, these produce feature maps of 32×32 , 16×16 , and 8×8 . Nuclei in H&E histopathology at 0.25 MPP typically measure 7 to 20 μm in diameter (28 to 80 pixels), making them small and densely packed relative to general objects. The P5 level at 8×8 resolution is too coarse to localize individual nuclei, and even P3 at 32×32 may miss the smallest cells. RayCastED therefore shifts the active pyramid levels to P2 through P4 (strides 4, 8, and 16), yielding a finest feature map of 64×64 for precise centroid localization. Extending to P2 through P5 is possible but adds anchor positions and backbone complexity that are redundant for nuclei-sized objects, which rarely require the receptive field of P5 ?? [TODO].

Standard downsampling operations (strided convolution and max pooling) act as low-pass filters that discard high-frequency content during feature map reduction. For large objects, this is acceptable because discriminative features reside in coarse shape and texture. For small nuclei, however, much of the discriminative signal lies in fine edges, chromatin texture, and boundary transitions that are smoothed away during standard downsampling ?? [TODO]. Discrete Wavelet Transform (DWT) based downsampling addresses this by decomposing the feature map into LL (approximation) and LH, HL, HH (detail) subbands. The LL subband serves as the downsampled representation while the detail subbands are preserved rather than discarded, retaining high-frequency information that would otherwise be lost ?? [TODO].

The P2 through P4 configuration generates approximately 5,376 anchor locations for a 256×256 input ($64^2 + 32^2 + 16^2$). A typical histopathology tile contains on the order of 100 nuclei, yielding a foreground-to-background ratio of roughly 100/5,376 ($\approx 1.9\%$). A single classification head forced to simultaneously discriminate foreground from background and classify cell types under such extreme imbalance produces weak gradients for both tasks. RayCastED addresses this by splitting classification into a binary head (foreground/background, 1 channel) trained on all anchors and a multiclass head (cell type, n_c channels) trained only on foreground anchors. At inference, the final score is $\sigma(\text{binary}) \times \text{softmax}(\text{class})$, ensuring that only confident foreground predictions contribute to class probabilities.

Dual-label assignment ?? [TODO] eliminates NMS by enforcing one-to-one matching within each pyramid level, but does not address cross-scale duplicates: the same nucleus may produce confident predictions at two adjacent pyramid levels simultaneously ?? [TODO]. Conventional NMS is unsuitable for resolving these duplicates in dense nuclei,

as valid detections of adjacent cells are indistinguishable from duplicate predictions based on overlap alone ?? [TODO]. Inter-scale competition complements dual-label assignment by normalizing classification scores across the scale dimension via softmax before output decoding. This operation suppresses lower-confidence cross-scale predictions during both training and inference in a fully differentiable manner, without requiring a hand-tuned IoU threshold.

3.2.2.2 Proposed Architecture Components

3.2.2.2.1 ResoConv.

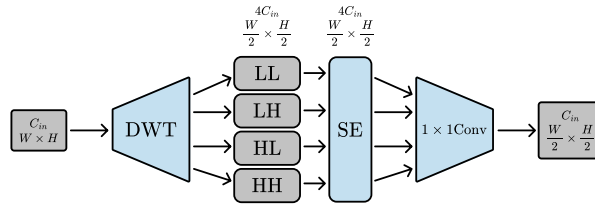


Figure 3.9. ResoConv

ResoConv replaces each strided downsampling step in the backbone and neck with a Haar DWT decomposition. The transform is implemented as a depthwise convolution with stride 2, where the four Haar filters are pre-computed and registered as non-trainable buffers. Depthwise convolution is used here for two reasons. First, the wavelet decomposition is inherently a per-channel operation: each input channel represents an independent feature map that should be decomposed independently, without cross-channel mixing during the transform itself. Cross-channel combination is deferred to the subsequent 1×1 projection. Second, depthwise convolution reduces the computational cost from $K^2 \cdot C^2$ to $K^2 \cdot C$ per spatial position ?? [TODO], which is significant given that ResoConv replaces every downsampling layer in both the backbone and neck. Unlike standard strided convolution, which acts as a learned low-pass filter and discards high-frequency content irreversibly, the DWT preserves this information explicitly across the four subbands ?? [TODO].

The four subbands are concatenated along the channel dimension and passed through a Squeeze-and-Excitation (SE) channel attention module. SE performs global average pooling followed by a bottleneck fully-connected layer, a ReLU activation, a restoring layer, and a sigmoid gate that produces per-channel weights. This allows the network to learn the relative importance of each subband at each layer: in early layers, high-frequency detail subbands may receive higher weights for boundary delineation, while in deeper layers, the LL approximation may dominate for semantic feature extraction.

The SE-reweighted features are then projected to the target channel count via a 1×1 convolution, producing the final output at half spatial resolution.

3.2.2.2.2 RayCastDetect.

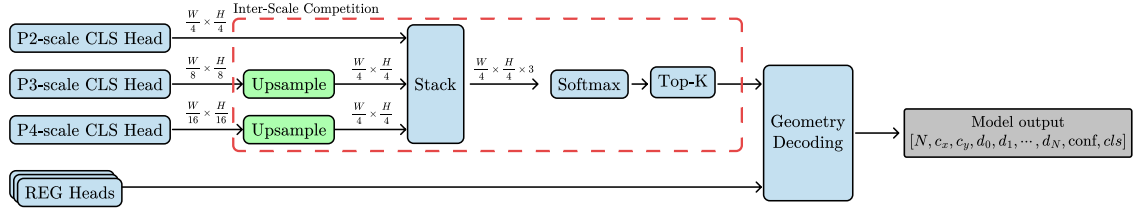


Figure 3.10. RayCastDetect

RayCastDetect extends the YOLO detection head to predict star-convex polygon parameters instead of axis-aligned bounding boxes. For each anchor, the regression branch produces $2 + R$ values: two centroid offsets and $R = 64$ ray distances radiating from the centroid at evenly spaced angles. The classification branch is split into a binary foreground/background head and a multiclass head with n_c channels, yielding a total of $3 + R + n_c$ output values per anchor. The head is trained with dual-label assignment ?? [TODO] using a one-to-many (o2m) head for rich supervision and a one-to-one (o2o) head for NMS-free inference; the o2m head is discarded after training. Three key design choices distinguish RayCastDetect from the standard YOLO head: inter-scale competition to suppress cross-scale duplicate detections, a split classification head to address foreground-background imbalance, and a regression head with RayRefinementBlock to smooth spatially inconsistent ray predictions.

Inter-scale Competition. Operating across three feature scales (P2, P3, P4), the head often assigns high confidence to the same nucleus at multiple scales simultaneously, producing cross-scale duplicate detections. Inter-scale competition addresses this by upsampling the classification scores from all scales to P2 resolution, applying softmax across the scale dimension, and multiplying each scale’s confidence by its resulting competition weight. This mechanism is fully differentiable and operates during both training and inference, providing a learned, soft suppression that complements (rather than replaces) the dual-label assignment strategy ?? [TODO]. Unlike NMS ?? [TODO], which requires a hand-tuned IoU threshold and is applied post-hoc, inter-scale competition is integrated into the network and suppresses duplicates through gradient-descent-optimized weights.

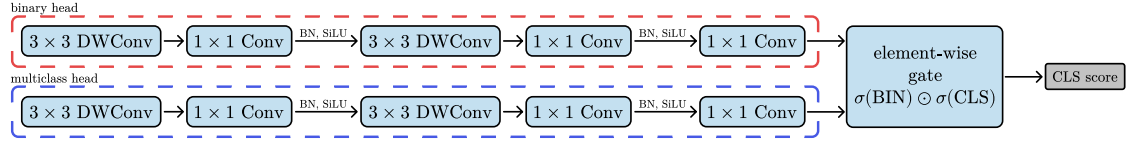


Figure 3.11. Split Classification Head

Split Classification Head. The P2-P4 scale configuration at 256×256 pixel input produces approximately 5,376 anchors per image, yet a typical histopathology image contains fewer than 100 nuclei. This extreme foreground-background imbalance makes joint objectness and classification difficult for a single head. RayCastDetect therefore splits the classification branch into a binary head with one channel that is applied to all anchors, and a multiclass head with n_c channels that is supervised only on foreground anchors. At inference, the two are combined as $P(\text{fg}) \times P(\text{type} \mid \text{fg})$, i.e., $\text{sigmoid}(\text{binary}) \times \text{softmax}(\text{class})$. This decomposition separates the easier binary problem of distinguishing nucleus from background, where the binary head can exploit all anchor signals, from the harder multiclass problem of cell-type discrimination, where the class head focuses exclusively on foreground samples.

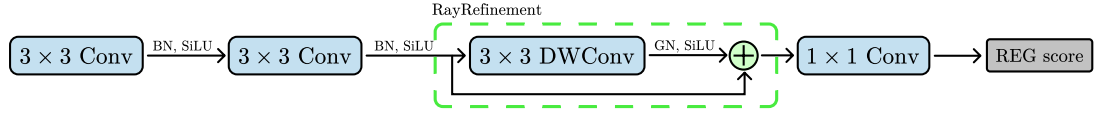


Figure 3.12. Regression Head with RayCast Refinement

Regression Head. The regression branch produces the polygon parameters per anchor through two 3×3 convolution layers, a RayRefinementBlock, and a final 1×1 convolution that outputs $2 + R = 66$ channels: two centroid offsets and 64 ray distances. The RayRefinementBlock applies a 3×3 depthwise convolution followed by GroupNorm and SiLU activation with a residual skip connection. GroupNorm is preferred over BatchNorm because each scale level (P2, P3, P4) maintains independent head weights, including its own RayRefinementBlock, and the three scales operate at different spatial resolutions (64×64 , 32×32 , 16×16) with different object distributions: P2 densely covers small nuclei, while P4 sparsely covers larger structures. BatchNorm would maintain separate running mean and variance statistics per scale computed over these inconsistent conditions, leading to unstable normalization when the statistics do not transfer cleanly. GroupNorm normalizes within channel groups per sample, avoiding any cross-scale or batch-dimension dependency. The residual connection ensures the block refines rather

than replaces the existing features: since the purpose is to smooth spatially inconsistent ray predictions by blending neighboring anchor features, the skip connection guarantees that the base prediction is preserved when no correction is needed. The two centroid channels use sigmoid activation to produce bounded $[0, 1]$ grid-cell offsets, while the 64 ray channels use softplus activation to ensure strictly positive distances. Bias initialization sets the ray channels to the minimum plausible nucleus size using the formula $(d_{\min} + \epsilon)/(2 \cdot S \cdot \rho)$, where $d_{\min} = 3.0 \mu\text{m}$ is the minimum nucleus diameter, $\epsilon = 0.5 \mu\text{m}$ is a linear approximation buffer, $S = 256$ is the crop size in pixels, and $\rho = 0.25 \mu\text{m}/\text{pixel}$ is the scanning resolution. This yields a normalized target of $3.5/128 \approx 0.027$, corresponding to a 7 pixel radius at $0.25 \mu\text{m}/\text{pixel}$. The inverse softplus of this value is used as the bias so that $\text{softplus}(\text{bias})$ produces the target directly at initialization, providing a unidirectional expansion gradient at the start of training.

3.2.3 Training Strategy and Optimization

3.2.3.1 Curriculum

Training uses the MuSGD optimizer, which applies the Muon optimizer ?? [TODO] to two-dimensional weight matrices (convolution kernels, linear layers) and standard SGD with Nesterov momentum to one-dimensional parameters (biases, normalization scales). This hybrid leverages Muon’s orthogonalized gradient updates for the high-rank weight matrices that dominate model capacity while retaining SGD stability for scalar parameters. The base learning rate is 0.01, momentum 0.937, weight decay 5×10^{-4} , and gradients are clipped to a maximum norm of 10.0. Training uses a batch size of 16 on 256×256 pixel crops.

The learning rate follows cosine annealing with warm restarts (SGDR) [?], with initial cycle length $T_0 = 133$ epochs, cycle multiplier $T_{\text{mult}} = 1$, and minimum learning rate $\eta_{\min} = 10^{-4}$. This produces three equal cycles over the full training run. Each restart abruptly increases the learning rate, helping the optimizer escape sharp minima and explore broader regions of the loss landscape before annealing again.

Three annealing schedules progressively shift the training objective. First, the o2m and o2o loss weights transition linearly from 0.8/0.2 to 0.1/0.9, gradually shifting from dense one-to-many supervision to clean one-to-one matching for NMS-free inference ?? [TODO]. Second, the o2o assigner’s topk2 parameter anneals from 3 to 1 (one-to-few to one-to-one), beginning at epoch 67 and completing at 50% of the training schedule, allowing multiple positive anchors per ground truth during early feature learning before converging to the strict bipartite matching required at inference. Third, the smoothness loss weight λ_{smooth} ramps from 0 to 3 over the first 40% of epochs, letting the ray predictions fit target shapes before enforcing boundary regularization.

3.2.3.2 Assignment

RayCastDetect uses two parallel instances of RayCastAssigner, which extends the task-aligned matching ?? [TODO] strategy by replacing standard IoU with PolarIoU in the alignment metric. The one-to-many branch uses $\text{topk} = 15$ and $\text{topk}_2 = 15$ for dense multi-anchor supervision, while the one-to-one branch uses $\text{topk} = 7$ (candidate pool) and $\text{topk}_2 = 3 \rightarrow 1$ (annealed). The o2m alignment metric is:

$$t = s^\alpha \cdot \text{PolarIoU}^\beta \cdot \exp\left(\frac{-d^2}{2\sigma^2}\right) \quad (3-25)$$

where s is the classification score, $\alpha = 0.5$, $\beta = 2.0$, d is the centroid-to-anchor distance, and σ is the per-nucleus radius computed as the 75th-percentile non-zero ray distance multiplied by a scale factor of 1.0. The inclusion of PolarIoU in the o2m metric ensures that early training prioritizes geometric quality: anchors are rewarded for producing well-shaped polygons, not just confident classifications. The Gaussian spatial decay term $\exp(-d^2/2\sigma^2)$ restricts high alignment scores to spatially proximate anchors, with σ adapted to each nucleus size.

The o2o branch uses classification-only task-aligned matching ($\text{cls}^\alpha \cdot \exp(-d^2/2\sigma^2)$, without PolarIoU), forcing the classification head to become the primary ranking signal. This matches inference conditions, where no localization feedback is available to resolve assignment ambiguity, and creates a self-reinforcing dynamic: higher classification confidence yields better assignments, which in turn produce stronger classification gradients.

3.2.3.3 Loss Function

The total training loss combines five terms:

$$\mathcal{L} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{xy}} \mathcal{L}_{\text{xy}} + \lambda_{\text{L1}} \mathcal{L}_{\text{L1}} + \lambda_{\text{piou}} \mathcal{L}_{\text{piou}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}} \quad (3-26)$$

with weights $\lambda_{\text{cls}} = 2.0$, $\lambda_{\text{xy}} = 500.0$, $\lambda_{\text{L1}} = 14.0$, $\lambda_{\text{piou}} = 3.0$, and λ_{smooth} annealed from 0 to 3 over the first 40% of training. Each branch (o2m and o2o) computes this loss independently, and the two are combined using the o2m/o2o weight schedule described above.

The classification loss differs between branches. The o2m head uses focal loss ($\gamma = 2.0$, $\alpha = 0.75$) with background subsampling at a 3:1 ratio of background to foreground anchors. The o2o head, using the split classification design, applies focal loss ($\gamma = 2.0$, $\alpha = 0.75$) on the binary foreground/background head across all anchors, and binary cross-entropy on the multiclass head restricted to foreground anchors only.

The ray distance loss $\mathcal{L}_{\text{L1}}$ is the most distinctive component. During data ingestion,

ground-truth ray distances are computed by casting rays from the annotation centroid to the polygon boundary using geometric line intersection. During training, this same raycasting operation is reformulated as a differentiable Cramer’s rule solver that computes analytical ray distances from the *predicted* centroid (with gradients detached) to the ground-truth polygon boundary. The loss is then evaluated in log-space:

$$\mathcal{L}_{L1} = \frac{1}{R} \sum_{i=1}^R \left| \log(\hat{d}_i + \varepsilon) - \log(d_i^{\text{analytical}} + \varepsilon) \right| \quad (3-27)$$

This analytical recomputation decouples centroid accuracy from ray supervision: the ray head is always judged against geometrically correct targets for the position where the model actually placed the centroid, rather than against static targets computed from the ground-truth centroid. If the predicted centroid shifts from the ground-truth centroid, the target rays adapt accordingly, preventing conflicting gradient signals between the centroid and ray branches. Log-space evaluation provides scale-invariant gradients, applying equal relative penalty for over-prediction and under-prediction regardless of absolute ray length.

The centroid loss $\mathcal{L}_{xy}$ uses Huber loss ($\delta = 0.05$) on normalized grid-cell offsets. The high weight $\lambda_{xy} = 500$ compensates for gradient attenuation caused by the sigmoid activation, stride multiplication, and pixel-space normalization that the offsets undergo during decoding. The PolarIoU loss $\mathcal{L}_{\text{pious}} = -\log(\text{PolarIoU})$ operates on static ground-truth rays (not analytical) and provides shape-aware geometric supervision through sector-area decomposition, where each sector area is $\frac{1}{2} \sin(2\pi/N) d_i d_{i+1}$, coupling adjacent rays through shared area computation. The smoothness loss $\mathcal{L}_{\text{smooth}}$ penalizes second-order circular curvature $|d_{i+1} - 2d_i + d_{i-1}|$ to regularize boundary roughness, with its weight annealed from 0 to 3 as described in the curriculum.

3.2.4 Evaluation Procedure

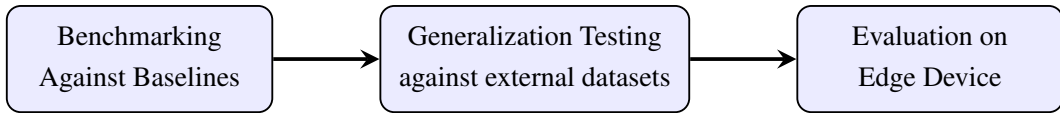


Figure 3.13. Evaluation Procedure Diagram

3.2.4.1 Evaluation Metrics

Predicted star-convex polygons are rasterized to binary masks for overlap-based evaluation. When multiple predicted masks overlap at the same pixel, the largest mask receives priority, following the LSP-DETR evaluation protocol to ensure fair comparison of methods that predict overlapping nuclei boundaries. All PQ-family metrics use Hungarian matching between predicted and ground-truth masks at $\text{IoU} \geq 0.5$. The F1 score, precision,

and recall use centroid-based matching at a distance threshold of 12 pixels, and AJI uses its own global intersection-over-union formulation ?? [TODO]. During training validation, bPQ is computed from PolarIoU (analytical, no rasterization required) as a fast proxy; final evaluation uses rasterized mask IoU for accuracy.

Three categories of metrics are reported. Detection quality is measured by F1, precision, recall, and the total number of predicted nuclei (indicating over- or under-segmentation tendency). Segmentation quality is measured by bPQ (binary, all classes merged), mPQ (per-class, averaged over the five PanNuke nuclei types), and AJI (global instance-level). Efficiency is measured by GFLOPs, parameter count, and runtime in milliseconds per image. The model selection fitness during training is $0.7 \times \text{bPQ} + 0.3 \times \text{mAP}_{50}$, following the Ultralytics convention where mAP at IoU 0.5 provides detection-quality feedback alongside the segmentation-focused bPQ.

3.2.4.2 Benchmarking

RayCastED is trained on PanNuke Fold 1, validated on Fold 2, and tested on Fold 3. The Base (B) variant is compared against StarDist [?], LKCell [24], HoVer-NeXt [?], and CellPose-SAM. All baselines are trained and evaluated on the same PanNuke fold split under their respective published configurations. A confidence threshold of 0.20 is used for all RayCastED predictions to test the model’s raw ranking quality rather than relying on aggressive thresholding to suppress false positives. All metrics described above are reported in a comprehensive comparison table.

3.2.4.3 Generalization Test

To assess cross-dataset generalization, RayCastED trained on PanNuke Fold 1 is evaluated without fine-tuning on MoNuSAC [?] and PUMA. Only AJI, bPQ, and F1 are reported to avoid dataset taxonomy clashes: MoNuSAC defines four nuclei classes and PUMA uses a different class schema, making per-class mPQ comparisons across datasets misleading. Class labels from target datasets are harmonized to PanNuke’s taxonomy where unambiguous mappings exist; where mappings are unclear, evaluation collapses to binary foreground/background. Results indicate whether features learned across PanNuke’s 19 tissue types transfer to unseen tissues, staining protocols, and scanning resolutions.

3.2.4.4 Evaluation on Edge Device

Two runtime backends are benchmarked on the desktop GPU (RTX 5070 Ti): native PyTorch and TensorRT. The PyTorch runtime serves as the accuracy reference and the TensorRT runtime establishes the performance ceiling achievable on a high-power GPU before edge deployment.

For edge deployment, the model is exported to ONNX (opset 17, graph simplified) and deployed on a Jetson Orin Nano. The ONNX graph exports raw logits only; all decoding (sigmoid for centroid offsets, softplus for ray distances, grid decoding, hierarchical classification combination) and postprocessing (top-k selection, confidence filtering, polygon scaling) are performed in a Python layer on-device. Three edge runtime configurations are evaluated: ONNX Runtime, TensorRT FP32, and TensorRT FP16. All configurations are tested on PanNuke Fold 3 for accuracy parity with the desktop results. Runtime is measured via wall-clock timing over all test images, reported as milliseconds per image. All metrics except GFLOPs and parameter count are reported, since the architecture is identical across backends. The FP32 versus FP16 comparison quantifies the accuracy-speed trade-off of half-precision inference on edge hardware.

3.3 Research Workflow

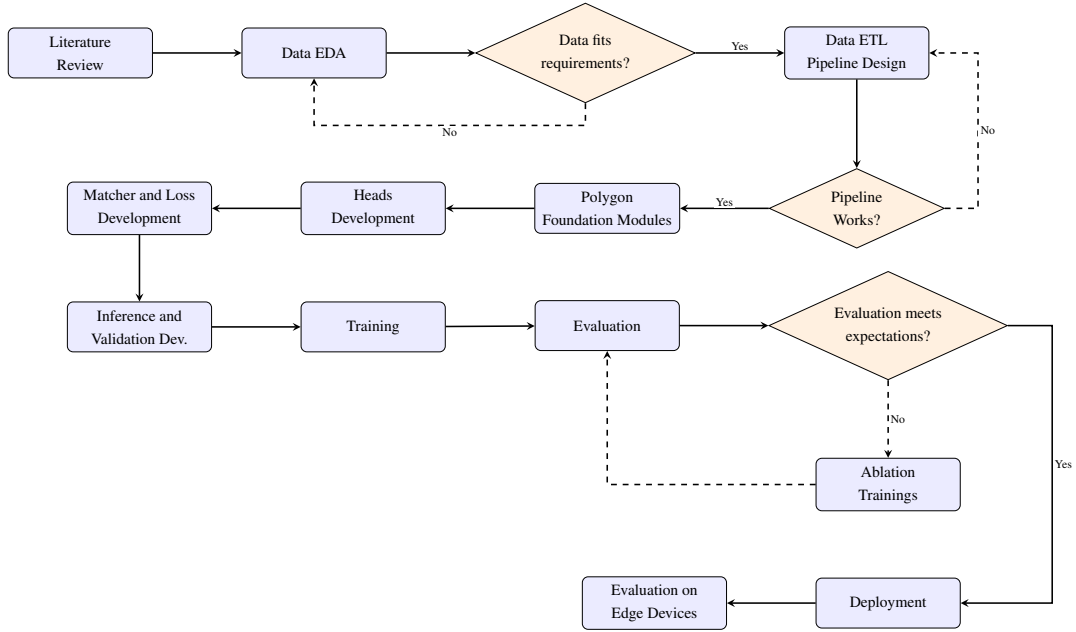


Figure 3.14. Research workflow diagram illustrating the iterative development process of the RayCastED model, from literature review and data preparation through model development, training, evaluation, and deployment.

CHAPTER IV

HASIL DAN PEMBAHASAN

Table 4.1. Evaluation Against LSP-DETR

Metric	LSP-DETR	RayCastED
F1	0.825	0.748
AJI	0.677	0.546
Precision	0.862	0.7618
Recall	0.791	0.7348
bPQ	0.657	0.5769
mPQ	0.476	0.4381
Params	45.0M	2.99M
Inference Time	-	3.75ms

Berikut ini adalah yang perlu diperhatikan untuk mengisi bab hasil dan pembahasan:

1. Setiap rumusan masalah boleh memiliki lebih dari 1 tujuan.
2. Setiap subbab harus spesifik menjawab setiap tujuan yang dituliskan.
3. Setiap rumusan masalah boleh dijawab dengan 1 subbab atau lebih.

Berikut ini adalah contoh sub bab untuk menjelaskan tujuan penelitian.

4.1 Pembahasan Tujuan 1 dengan Hasil Penelitian 1 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)

Sub bab pertama adalah membahas tujuan penelitian pertama dengan hasil penelitian ke-1. Dapat ditambahkan beberapa sub bab jika diperlukan.

4.2 Pembahasan Tujuan 1 dengan Hasil Penelitian 2 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)

Sub bab kedua adalah membahas tujuan penelitian pertama dengan hasil penelitian ke-2. Sub bab ini merupakan contoh tambahan sub bab pertama.

4.3 Pembahasan Tujuan 2 dengan Hasil Penelitian 3 (Ubah Judul Sesuai dengan Hal yang Hendak dibahas)

Sub bab ketiga adalah membahas tujuan penelitian kedua. Dapat ditambahkan beberapa sub bab jika diperlukan.

4.4 Perbandingan Hasil Penelitian dengan Hasil Terdahulu

Pembahasan penutup dapat menjelaskan mengenai kelebihan hasil pengembangan / penelitian dan kekurangan dibandingkan dengan skripsi atau penelitian terdahulu atau perbandingan terhadap produk lain yang ada di pasaran. Penulis dapat menggunakan tabel untuk membandingkan secara gamblang dan menjelaskannya.

CHAPTER V

TAMBAHAN (OPSIONAL)

Anda boleh menambahkan Bab jika diperlukan. Jumlah Bab tidak harus sesuai dengan *template*.

Bab tambahan ini diperlukan jika hasil penelitian untuk menjawab tujuan cukup panjang atau terdiri dari banyak sub bab. Mahasiswa boleh menjawab 1 tujuan penelitian dengan 1 bab.

CHAPTER VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Kesimpulan dapat diawali dengan apa yang dilakukan dengan tugas akhir ini lalu dilanjutkan dengan poin-poin yang menjawab tujuan penelitian, apakah tujuan sudah tercapai atau belum, tentunya berdasarkan data ataupun hasil dari Bab pembahasan sebelumnya. Dalam beberapa hal, kesimpulan dapat juga berisi tentang temuan/*findings* yang Anda dapatkan setelah melakukan pengamatan dan atau analisis terhadap hasil penelitian.

Kesimpulan menjawab seberapa jauh rumusan masalah tercapai berdasarkan hasil penelitian. Semua rumusan masalah harus disimpulkan berdasarkan data penelitian.

6.2 Saran

Saran berisi hal-hal yang bisa dilanjutkan dari penelitian atau skripsi ini, yang belum dilakukan karena batasan permasalahan. Saran bukan berisi saran kepada sistem atau pengguna, tetapi saran diberikan kepada aspek penelitian yang dapat dikembangkan dan ditambahkan di penelitian atau skripsi selanjutnya.

Catatan: Mahasiswa perlu melihat sinkronisasi antara rumusan masalah, tujuan, metode, hasil penelitian, dan kesimpulan.

REFERENCES

- [1] A. Basu, P. Senapati, M. Deb, R. Rai, and K. G. Dhal, “A survey on recent trends in deep learning for nucleus segmentation from histopathology images,” *Evolving Systems*, vol. 15, no. 1, pp. 203–248, 2024.
- [2] S. Deng, X. Zhang, W. Yan, E. I.-C. Chang, Y. Fan, M. Lai, and Y. Xu, “Deep learning in digital pathology image analysis: a survey,” *Frontiers of Medicine*, vol. 14, no. 4, pp. 470–487, 2020.
- [3] A. J. Shephard, S. Graham, R. M. S. Bashir, M. Jahanifar, H. Mahmood, S. A. Khurram, and N. M. Rajpoot, “Simultaneous nuclear instance and layer segmentation in oral epithelial dysplasia,” *arXiv preprint arXiv:2108.13904*, 2021.
- [4] J. D. Nunes, D. Montezuma, D. Oliveira, T. Pereira, and J. S. Cardoso, “A survey on cell nuclei instance segmentation and classification: Leveraging context and attention,” *Medical Image Analysis*, vol. 99, p. 103360, 2025.
- [5] R. Zhang, Z. Tian, C. Shen, M. You, and Y. Yan, “Mask encoding for single shot instance segmentation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 223–10 232.
- [6] S. Eppel, H. Xu, M. Bismuth, and A. Aspuru-Guzik, “Computer vision for recognition of materials and vessels in chemistry lab settings and the vector-labpics data set,” *ACS Central Science*, vol. 6, no. 10, pp. 1743–1752, 2020.
- [7] E. Xie, P. Sun, X. Song, W. Wang, D. Liang, C. Shen, and P. Luo, “Polarmask: Single shot instance segmentation with polar representation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 12 193–12 202.
- [8] U. Schmidt, M. Weigert, C. Broaddus, and E. Myers, “Cell detection with star-convex polygons,” in *MICCAI*, 2018, pp. 265–273.
- [9] M. Weigert and U. Schmidt, “Nuclei instance segmentation and classification in histopathology images with stardist,” in *IEEE ISBIC*, 2022.
- [10] S. Chen, C. Ding, M. Liu, J. Cheng, J. Yang, and D. Tao, “Cpp-net: Context-aware polygon proposal network for nucleus segmentation,” *IEEE Transactions on Image Processing*, vol. 32, pp. 980–994, 2023.
- [11] E. Baumann, B. Dislich, J. L. Rumberger, M. J. Abdolhosseini Qomi, and T. J. Fuchs, “Hover-next: A fast nuclei segmentation and classification pipeline for next generation histopathology,” in *Medical Imaging with Deep Learning (MIDL)*, 2024.
- [12] S. Graham, Q. D. Vu, S. E. A. Raza *et al.*, “Hover-net: Simultaneous segmentation and classification of nuclei in multi-tissue histology images,” *Medical Image Analysis*, vol. 58, p. 101563, 2019.
- [13] C. Stringer, T. Wang, M. Michaelos, and M. Pachitariu, “Cellpose: a generalist algorithm for cellular segmentation,” *Nature Methods*, vol. 18, pp. 100–106, 2021.

- [14] C. Stringer and M. Pachitariu, “Cellpose-sam: Superhuman generalization for cellular segmentation,” *bioRxiv*, 2025.
- [15] M. Pekár, V. Musil, R. Nenutil, P. Holub, and T. Brázdil, “Lsp-detr: Efficient and scalable nuclei segmentation in whole slide images,” *arXiv preprint arXiv:2601.03163*, 2026.
- [16] R. Salgado, C. Denkert, S. Demaria, N. Saine, S. Loi *et al.*, “The evaluation of tumor-infiltrating lymphocytes (tils) in breast cancer: recommendations by an international tils working group 2014,” *Annals of Oncology*, vol. 26, no. 2, pp. 259–271, 2015.
- [17] F. Pagès, B. Mlecnik, F. Marliot, G. Bindea, F. S. Ou, C. Bifulco, A. Lugli, I. Zlobec, T. T. Rau, M. D. Berger *et al.*, “International validation of the consensus immunoscore for the classification of colon cancer: a prognostic and accuracy study,” *The Lancet*, vol. 391, no. 10135, pp. 2128–2139, 2018.
- [18] M. Macenko, M. Niethammer, J. S. Marron, D. Borland, J. T. Woosley, X. Guan, P. Schmitt, and N. E. Thomas, “A method for normalizing histology slides for quantitative analysis,” in *ISBI*. IEEE, 2009, pp. 1107–1110.
- [19] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-cnn,” in *ICCV*, 2017, pp. 2961–2969.
- [20] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, 2015, pp. 234–241.
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *CVPR*, 2016, pp. 779–788.
- [22] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” in *NeurIPS*, 2015, pp. 91–99.
- [23] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [24] Z. Cui, J. Yao, L. Zeng, J. Yang, W. Liu, and X. Wang, “Lkcell: Efficient cell nuclei instance segmentation with large convolution kernels,” *arXiv preprint arXiv:2407.18054*, 2024.
- [25] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” 2017.
- [26] I. Daubechies, *Ten Lectures on Wavelets*, ser. CBMS-NSF Regional Conference Series in Applied Mathematics. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 1992, vol. 61.
- [27] S. G. Mallat, “A theory for multiresolution signal decomposition: The wavelet representation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 674–693, 1989.

- [28] Y. Zhang, Y. Fang, F. Liu, Z. Liu, and X. Jia, "Waveconvx: Multi-level wavelet enhancement for histopathology image classification," in *IEEE International Conference on Systems, Man and Cybernetics (SMC)*, 2025.
- [29] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. New York: Springer-Verlag, 1985.
- [30] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollar, "Panoptic segmentation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9404–9413.
- [31] J. Gamper, N. A. Koohbanani, S. Graham, M. Jahanifar, S. A. Khurram, A. Azam, K. Hewitt, and N. Rajpoot, "Pannuke dataset extension, insights and baselines," *arXiv preprint arXiv:2003.10778*, 2020.
- [32] S. Graham, M. Jahanifar, A. Azam, N. Nimir, N. Tsiknakis, M. Zerizer, D. Hamam, N. A. Koohbanani, and N. Rajpoot, "Consep: A dataset for addressing cellular semantic segmentation and phenotype classification in histology images," in *European Congress on Digital Pathology*. Springer, 2021, pp. 49–60.
- [33] N. Kumar, R. Verma, S. Sharma, S. Bhargava, A. Vahadane, and A. Sethi, "A dataset and a technique for generalized nuclear segmentation for computational pathology," in *IEEE Transactions on Medical Imaging*, vol. 36, no. 7, 2017, pp. 1550–1560.
- [34] J. Gamper, N. A. Koohbanani, K. Benes, A. Khuram, and N. Rajpoot, "Pannuke: an open pan-cancer histology dataset for nuclei instance segmentation and classification," in *European Congress on Digital Pathology*. Springer, 2019, pp. 11–19.
- [35] M. Schuiveling, "Melanoma histopathology dataset with tissue and nuclei annotations," Mar. 2025.
- [36] S. Liu, M. Amgad, M. Rathore, R. Salgado, and L. Cooper, "A panoptic segmentation approach for tumor-infiltrating lymphocyte assessment: development of the mutils model and panoptils dataset," *medRxiv*, 2022.
- [37] R. Verma, N. Kumar, A. Patil, N. C. Kurian, S. Rane, S. Graham *et al.*, "Monusac2020: A multi-organ nuclei segmentation and classification challenge," *IEEE Transactions on Medical Imaging*, vol. 40, no. 12, pp. 3413–3423, 2021.
- [38] J. L. Agraz, J. Grewal, L. Montaser-Kouhsari, H. M. Kluger, S. Halene, and Y. Kluger, "Robust image population based stain color normalization: How many reference slides are enough?" *IEEE Open Journal of Engineering in Medicine and Biology*, vol. 3, pp. 218–226, 2023.
- [39] L. E. Nugroho, "E-book as a platform for exploratory learning interactions," *International Journal of Emerging Technologies in Learning (iJET)*, vol. 11, no. 01, pp. 62–65, 2016. [Online]. Available: <http://www.online-journals.org/index.php/i-jet/article/view/5011>
- [40] P. I. Santosa, "User?s preference of web page length," *International Journal of Research and Reviews in Computer Science*, pp. 180–185, 2011.

- [41] N. A. Setiawan, “Fuzzy decision support system for coronary artery disease diagnosis based on rough set theory,” *International Journal of Rough Sets and Data Analysis (IJRSDA)*, vol. 1, no. 1, pp. 65–80, 2014.
- [42] C. P. Wibowo, P. Thumwarin, and T. Matsuura, “On-line signature verification based on forward and backward variances of signature,” in *Information and Communication Technology, Electronic and Electrical Engineering (JICTEE), 2014 4th Joint International Conference on.* IEEE, 2014, pp. 1–5.
- [43] D. A. Marenda, A. Nasikun, and C. P. Wibowo, “Digitory, a smart way of learning islamic history in digital era,” *arXiv preprint arXiv:1607.07790*, 2016.
- [44] S. Wibirama, S. Tungjitkusolmun, and C. Pintavirooj, “Dual-camera acquisition for accurate measurement of three-dimensional eye movements,” *IEEJ Transactions on Electrical and Electronic Engineering*, vol. 8, no. 3, pp. 238–246, 2013.
- [45] C. P. Wibowo, “Clustering seasonal performances of soccer teams based on situational score line,” *Communications in Science and Technology*, vol. 1, no. 1, 2016.

Catatan: Daftar pustaka adalah apa yang dirujuk atau disitasi, bukan apa yang telah dibaca, jika tidak ada dalam sitasi maka tidak perlu dituliskan dalam daftar pustaka.

LAMPIRAN

L.1 Isi Lampiran

Lampiran bersifat opsional bergantung hasil kesepakatan dengan pembimbing dapat berupa:

1. Bukti pelaksanaan Kuesioner seperti pertanyaan kuesioner, resume jawaban responden, dan dokumentasi kuesioner.
2. Spesifikasi Aplikasi atau Sistem yang dikembangkan meliputi spesifikasi teknis aplikasi, tautan unduh aplikasi, manual penggunaan aplikasi, hingga screenshot aplikasi.
3. Cuplikan kode yang sekiranya penting dan ditambahkan.
4. Tabel yang terlalu panjang yang masih diperlukan tetapi tidak memungkinkan untuk ditayangkan di bagian utama skripsi.
5. Gambar-gambar pendukung yang tidak terlalu penting untuk ditampilkan di bagian utama. Akan tetapi, mendukung argumentasi/pengamatan/analisis.
6. Penurunan rumus-rumus atau pembuktian suatu teorema yang terlalu panjang dan terlalu teknis sehingga Anda berasumsi bahwa pembaca biasa tidak akan menelaah lebih lanjut. Hal ini digunakan untuk memberikan kesempatan bagi pembaca tingkat lanjut untuk melihat proses penurunan rumus-rumus ini.

LAMPIRAN

L.2 Panduan Latex

L.2.1 Syntax Dasar

L.2.1.1 Penggunaan Sitasi

Contoh penggunaan sitasi [39,40] [41] [42] [43] [44,45]

L.2.1.2 Penulisan Gambar

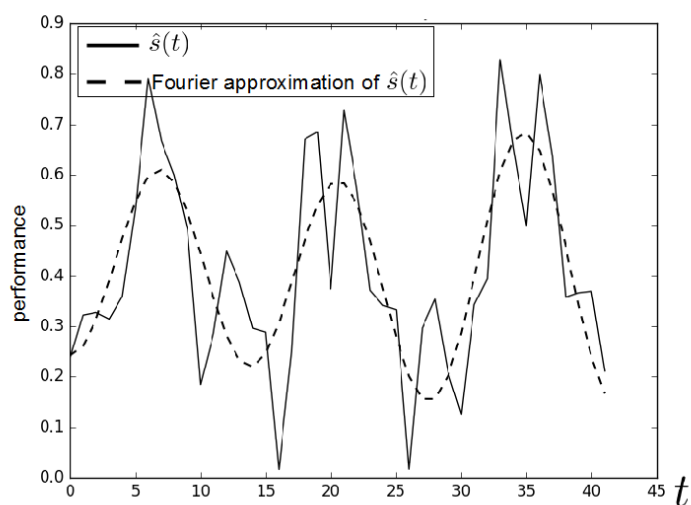


Figure L.1. Contoh gambar.

Contoh gambar terlihat pada Gambar L.1. Gambar diambil dari [45].

L.2.1.3 Penulisan Tabel

Table L.1. Tabel ini

ID	Tinggi Badan (cm)	Berat Badan (kg)
A23	173	62
A25	185	78
A10	162	70

Contoh penulisan tabel bisa dilihat pada Tabel L.1.

L.2.1.4 Penulisan formula

Contoh penulisan formula

$$L_{\psi_z} = \{t_i \mid v_z(t_i) \leq \psi_z\} \quad (1)$$

Contoh penulisan secara *inline*: $PV = nRT$. Untuk kasus-kasus tertentu, kita membutuhkan perintah "mathit" dalam penulisan formula untuk menghindari adanya jeda saat penulisan formula.

Contoh formula **tanpa** menggunakan "mathit": $PVA = RTD$

Contoh formula **dengan** menggunakan "mathit": $PVA = RTD$

L.2.1.5 Contoh list

Berikut contoh penggunaan list

1. First item
2. Second item
3. Third item

L.2.2 Blok Beda Halaman

L.2.2.1 Membuat algoritma terpisah

Untuk membuat algoritma terpisah seperti pada contoh berikut, kita dapat memanfaatkan perintah *algstore* dan *algrestore* yang terdapat pada paket *algcompatible*. Pada dasarnya, kita membuat dua blok algoritma dimana blok pertama kita simpan menggunakan *algstore* dan kemudian di-restore menggunakan *algrestore* pada algoritma kedua. Perintah tersebut dimaksudkan agar terdapat kesinamungan antara kedua blok yang sejatinya adalah satu blok.

Algorithm 1 Contoh algorima

```
1: procedure CREATESET( $v$ )  
2:   Create new set containing  $v$   
3: end procedure
```

Pada blok algoritma kedua, tidak perlu ditambahkan caption dan label, karena sudah menjadi satu bagian dalam blok pertama. Pembagian algoritma menjadi dua bagian ini berguna jika kita ingin menjelaskan bagian-bagian dari sebuah algoritma, maupun untuk memisah algoritma panjang dalam beberapa halaman.

```
4: procedure CONCATSET( $v$ )  
5:   Create new set containing  $v$   
6: end procedure
```

L.2.2.2 Membuat tabel terpisah

Untuk membuat tabel panjang yang melebihi satu halaman, kita dapat mengganti kombinasi *table* + *tabular* menjadi *longtable* dengan contoh sebagai berikut.

Table L.2. Contoh tabel panjang

header 1	header 2
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar

L.2.2.3 Menulis formula terpisah halaman

Terkadang kita butuh untuk menuliskan rangkaian formula dalam jumlah besar sehingga melewati batas satu halaman. Solusi yang digunakan bisa saja dengan memindahkan satu blok formula tersebut pada halaman yang baru atau memisah rangkaian formula menjadi dua bagian untuk masing-masing halaman. Cara yang pertama mungkin akan menghasilkan alur yang berbeda karena ruang kosong pada halaman pertama akan diisi oleh teks selanjutnya. Sehingga di sini kita dapat memanfaatkan *align* yang sudah diatur dengan mode *allowdisplaybreaks*. Penggunaan *align* ini memungkinkan satu rangkaian formula terpisah berbeda halaman.

Contoh sederhana dapat digambarkan sebagai berikut.

$$\begin{aligned}
 x &= y^2 \\
 x &= y^3 \\
 a + b &= c \\
 x &= y - 2 \\
 a + b &= d + e \\
 x^2 + 3 &= y \\
 a(x) &= 2x
 \end{aligned}
 \tag{2}$$

$$b_i = 5x$$

$$10x^2 = 9x$$

$$2x^2 + 3x + 2 = 0$$

$$5x - 2 = 0$$

$$d = \log x$$

$$y = \sin x$$

LAMPIRAN

L.3 Format Penulisan Referensi

Penulisan referensi mengikuti aturan standar yang sudah ditentukan. Untuk internasionalisasi DTETI, maka penulisan referensi akan mengikuti standar yang ditetapkan oleh IEEE (*International Electronics and Electrical Engineers*). Aturan penulisan ini bisa diunduh di <http://www.ieee.org/documents/ieeecitationref.pdf>. Gunakan Mendeley sebagai *reference manager* dan *export* data ke format Bibtex untuk digunakan di Latex.

Berikut ini adalah sampel penulisan dalam format IEEE:

L.3.1 Book

Basic Format:

- [1] J. K. Author, "Title of chapter in the book," in Title of His Published Book, xth ed. City of Publisher, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx-xxx.

Examples:

- [1] B. Klaus and P. Horn, Robot Vision. Cambridge, MA: MIT Press, 1986.
- [2] L. Stein, "Random patterns," in Computers and You, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.
- [3] R. L. Myer, "Parametric oscillators and nonlinear materials," in Nonlinear Optics, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.
- [4] M. Abramowitz and I. A. Stegun, Eds., Handbook of Mathematical Functions (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.
- [5] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.
- [6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), Integrated Electronic Systems. Englewood Cliffs, NJ: Prentice-Hall, 1970.
- [7] M. Gorkii, "Optimal design," Dokl. Akad. Nauk SSSR, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., The Mathematical Theory of Optimal Processes. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).
- [8] G. O. Young, "Synthetic structure of industrial plastics," in Plastics, vol. 3,

Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64.

L.3.2 Handbook

Basic Format:

[1] Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, year, pp. xx-xx.

Examples:

[1] Transmission Systems for Communications, 3rd ed., Western Electric Co., Winston Salem, NC, 1985, pp. 44-60.

[2] Motorola Semiconductor Data Manual, Motorola Semiconductor Products Inc., Phoenix, AZ, 1989.

[3] RCA Receiving Tube Manual, Radio Corp. of America, Electronic Components and Devices, Harrison, NJ, Tech. Ser. RC-23, 1992.

Conference/Prosiding

Basic Format:

[1] J. K. Author, "Title of paper," in Unabbreviated Name of Conf., City of Conf., Abbrev. State (if given), year, pp.xxx-xxx.

Examples:

[1] J. K. Author [two authors: J. K. Author and A. N. Writer] [three or more authors: J. K. Author et al.], "Title of Article," in [Title of Conf. Record as], [copyright year] © [IEEE or applicable copyright holder of the Conference Record]. doi: [DOI number]

Sumber Online/Internet

Basic Format:

[1] J. K. Author. (year, month day). Title (edition) [Type of medium]. Available: [http://www.\(URL\)](http://www.(URL))

Examples:

[1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: <http://www.atm.com>

Skripsi, Tesis dan Disertasi

Basic Format:

[1] J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

[2] J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

Examples:

[1] J. O. Williams, "Narrow-band analyzer," Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, 1993. [2] N. Kawasaki, "Parametric study of thermal and chemical nonequilibrium nozzle flow," M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993

LAMPIRAN

L.4 Contoh Source Code

L.4.1 Sample algorithm

Algorithm 2 Kruskal's Algorithm

```
1: procedure MAKESET( $v$ )
2:   Create new set containing  $v$ 
3: end procedure
4:
5: function FINDSET( $v$ )
6:   return a set containing  $v$ 
7: end function
8:
9: procedure UNION( $u, v$ )
10:  Unites the set that contain  $u$  and  $v$  into a new set
11: end procedure
12:
13: function KRUSKAL( $V, E, w$ )
14:   $A \leftarrow \{\}$ 
15:  for each vertex  $v$  in  $V$  do
16:    MakeSet( $v$ )
17:  end for
18:  Arrange  $E$  in increasing costs, ordered by  $w$ 
19:  for each  $(u, v)$  taken from the sorted list do
20:    if FindSet( $u$ )  $\neq$  FindSet( $v$ ) then
21:       $A \leftarrow A \cup \{(u, v)\}$ 
22:      Union( $u, v$ )
23:    end if
24:  end for
25:  return  $A$ 
26: end function
```

L.4.2 Sample Python code

```
1 import numpy as np
2
3 def incmatrix (genl1 , genl2):
4     m = len (genl1)
5     n = len (genl2)
6     M = None #to become the incidence matrix
7     VT = np.zeros ((n*m,1) , int) #dummy variable
8
9     #compute the bitwise xor matrix
10    M1 = bitxormatrix (genl1)
11    M2 = np.triu (bitxormatrix (genl2) ,1)
12
13    for i in range (m-1):
14        for j in range (i+1, m):
15            [r,c] = np.where (M2 == M1[i,j])
16            for k in range (len (r)):
17                VT[(i)*n + r[k]] = 1;
18                VT[(i)*n + c[k]] = 1;
19                VT[(j)*n + r[k]] = 1;
20                VT[(j)*n + c[k]] = 1;
21
22    if M is None:
23        M = np.copy (VT)
24    else:
25        M = np.concatenate ((M, VT) , 1)
26
27    VT = np.zeros ((n*m,1) , int)
28
29    return M
```

L.4.3 Sample Matlab code

```
1 function X = BitXorMatrix(A,B)
2 %function to compute the sum without charge of two vectors
3
4 %convert elements into unsigned integers
5 A = uint8(A);
6 B = uint8(B);
7
8 m1 = length(A);
9 m2 = length(B);
10 X = uint8(zeros(m1, m2));
11 for n1=1:m1
12     for n2=1:m2
13         X(n1, n2) = bitxor(A(n1), B(n2));
14     end
15 end
```